

Automated Detection of Satellite Trails in Ground-Based Observations Using U-Net and the Probabilistic Hough Transform

A REPORT PRESENTED BY

BARON GRACIAS

Departments

Department of Physics (Cavendish Laboratory)
Institute of Astronomy

Degree

MPhil Data Intensive Science

Supervisor

Dr Eduardo Gonzalez-Solares



CLARE COLLEGE
UNIVERSITY OF CAMBRIDGE

July 2026

Abstract

Automated Detection of Satellite Trails in Ground-Based Observations Using U-Net and the Probabilistic Hough Transform

Baron Gracias

The rapid growth of low-Earth-orbit satellite constellations increasingly contaminates wide-field survey images with linear artefacts. Automated satellite-trail detection and masking are becoming a requirement for observatories. This dissertation replicates the U-Net plus probabilistic Hough transform pipeline of Stoppa et al. [24] on a labelled MeerLICHT subset. The implementation uses an architecture-faithful U-Net, a combined binary cross-entropy and squared-Dice loss, validation-only model and threshold selection, and probabilistic Hough post-processing. It is examined through four areas: error analysis, model variants, training sensitivity, and inference-time strategies. The replication reproduces the paper’s recall, with test recall 0.923 against 0.94. Hough post-processing reproduces the paper’s completeness effect: trail-pixel recall rises from 0.918 to 0.990, and the pixel-level false-negative rate falls from 0.082 to 0.0105, comparable to the paper’s 6.98 % $\rightarrow$ 3.38 %. Pixel precision remains below the paper and is prevalence-dependent. The error analysis localises much of the gap near annotated trails: 82.5 % of false positives fall in trail-bearing patches; tolerance and distance checks place many within one pixel of the mask. A single-author re-annotation of 64 crops does not support systematic width bias between the original masks, the re-annotation, and the model. Model, training, and inference variants do not close the precision gap either. Altogether, the evidence supports a narrow interpretation: boundary-scale annotation disagreement is the best-supported explanation for the precision gap on this dataset. The practical implication is that high-recall masking can still be useful when pixel precision is boundary-limited, and that thin-artefact detectors should report pixel scores alongside boundary-tolerant checks and mask-width conventions.

Contents

1	Introduction and Context	4
1.1	The target pipeline	4
1.2	Background and related work	5
1.2.1	Classical trail detection	5
1.2.2	Deep learning for astronomical segmentation	5
1.2.3	The target pipeline: Stoppa et al. (2024)	5
1.3	Aims and scope	5
2	Dataset Characteristics	6
2.1	The MeerLICHT subset	6
2.2	Patch extraction and class imbalance	6
2.3	Dataset splits	8
2.4	Mask quality	8
3	Methodological Framing	10
3.1	Problem formulation	10
3.2	Segmentation with Hough post-processing	10
3.3	U-Net architecture choice	11
3.4	Combined BCE and Dice loss	11
3.5	Evaluation and assumptions	11
4	Model Implementation and Training	12
4.1	Data pipeline and augmentation	12
4.2	Network architecture	12
4.3	Loss function	13
4.4	Hyperparameter search	14
4.5	Threshold selection	14
4.6	Hough post-processing	14
4.7	Patch CNN classifier gate	15
4.8	Evaluation metrics	15

5	Replication and Error Diagnosis	16
5.1	Replication of the target detector	16
5.2	Validation-only model selection	16
5.3	Hough improves pixel completeness	18
5.4	Uncertainty quantification	19
5.5	Qualitative prediction checks	19
5.6	Precision gap near boundaries	21
5.6.1	False-positive decomposition	21
5.6.2	Boundary-tolerant evaluation	21
5.6.3	False-positive distance to mask	22
5.6.4	Relative display-contrast checks	23
5.6.5	Topology-aware comparison (clDice)	23
5.6.6	Re-annotation audit	24
5.7	Model variants as error checks	25
5.7.1	Classifier-gated detector	25
5.7.2	Attention-gated U-Net	25
5.8	Training and inference sensitivity	26
5.8.1	Data-side sensitivity	26
5.8.2	Soft labels and inference variants	27
6	Discussion	28
6.1	Differences from the target paper	28
6.2	Interpreting the headline comparison	29
6.3	Limitations	29
6.4	Scientific implications	30
7	Conclusions and Future Work	32
7.1	Conclusions	32
7.2	Future work	33
	Bibliography	34
A	Supplementary Results	38
A.1	Post-hoc Trail and Error Checks	38
A.2	Supplementary Tables	43
A.3	Prediction Galleries	43
B	Use of Auto-generation Tools	45

Chapter 1

Introduction and Context

The low-Earth-orbit satellite population is growing quickly. CelesTrak listed 15,848 active satellites on 27 May 2026, of which approximately 14,600 occupied orbits below 2,000 km, including more than 10,000 Starlink spacecraft [6]. Regulatory approvals extend further: 15,000 second-generation Starlink satellites are approved [9], and policy reviews treat proposals exceeding 100,000 spacecraft as realistic [29, 30]. Each sunlit satellite crossing a wide-field exposure leaves a bright, near-linear trail that corrupts photometry along its path and produces false-positive detections into transient-detection pipelines [10, 27].

This contamination is already measurable. Starlink trails affected under 0.5 % of Zwicky Transient Facility twilight images in late 2019 but 18 % by August 2021 [16]. Rubin Observatory simulations indicate that approximately 10 % of LSST images would contain at least one trail under a 40,000-satellite scenario [28]. Other wide-field twilight exposures can reach 30–40 % contamination [2, 10], and 4.3 ± 0.4 % of Hubble ACS/WFC images taken in 2018–2021 contained trails [13]. At these rates manual flagging does not scale: a survey instrument such as MeerLICHT [4] produces $10,560 \times 10,560$ -pixel frames, so identifying and masking affected pixels must be automated, reliable, and cheap enough to run along the reduction pipeline.

1.1 The target pipeline

Stoppa et al. [24] present ASTA, the pipeline replicated in this dissertation: a U-Net [20] trained on manually annotated MeerLICHT images, followed by a probabilistic Hough transform [12, 14] applied to the network’s output probability map. The two stages serve complementary roles. The U-Net supplies per-pixel soft detection that survives the crowded stellar background. The Hough stage then adds an explicit global-line constraint, connecting fragmented detections and extending partially recovered trails when the probability map already contains weak linear evidence. The pipeline reported pixel precision and recall of 0.94 and was applied to around 200,000 full MeerLICHT fields.

1.2 Background and related work

1.2.1 Classical trail detection

Classical satellite-trail detection treats trails as geometric line features. Hough and Radon-family methods map image pixels into line-parameter space, making them natural tools for long, straight artefacts [8, 12, 14]. Astronomical uses include survey cleaning, masked-star validation in crowded fields, faint-streak filtering, and Hubble ACS trail recovery [3, 17, 23, 25]. A common limitation is sensitivity to faint, fragmented, or background-confused trails, where reliable voting and instrument-specific preprocessing become difficult; this motivates learned segmentation before geometric post-processing.

1.2.2 Deep learning for astronomical segmentation

The U-Net architecture [20] is an encoder–decoder with skip connections that help recover fine spatial detail. It was designed for biomedical segmentation and is now common in astronomical imaging. Close precedents for thin, sparse artefacts are deepCR [33] and Cosmic-CoNN [31], both cosmic-ray segmentation systems. MaxiMask [19] covers contaminant masking including satellite and aircraft trails, and trails have since been targeted with U-Net plus line-segment detection [7], multi-band attention U-Nets [32], and end-to-end streak cataloguing [5]. No published trail-detection pipeline was found that gates a segmentation U-Net with a lightweight patch classifier. Section 5.7.1 explores this.

1.2.3 The target pipeline: Stoppa et al. (2024)

Stoppa et al. [24] trained a U-Net on 528×528 MeerLICHT patches with a combined binary cross-entropy and Dice loss, and applied a probabilistic Hough transform to the U-Net probability map as a post-processing step. On a 20,000-patch test set at threshold 0.58 the pipeline reported pixel precision and recall both approaching 0.94, with the Hough step reducing the pixel-level false-negative rate from 6.98 % to 3.38 %. The paper does not report the hyperparameters, making replication a non-trivial search (Chapter 4).

1.3 Aims and scope

This dissertation has three aims. The first is a replication of the Stoppa et al. [24] pipeline on the labelled MeerLICHT subset, with the network architecture reconstructed from the released Keras `model-best.h5` file. The second is to diagnose the error of that replication, asking how much of the precision gap can be explained by evaluation population, boundary sensitivity, and false positives. The third is to test which interventions can move that gap, through model/training variants or inference-time strategies.

Chapter 2

Dataset Characteristics

2.1 The MeerLICHT subset

MeerLICHT is a 65 cm wide-field optical telescope [4]. Its $10,560 \times 10,560$ -pixel CCD samples a 2.7 deg^2 field of view at $0.56'' \text{ pixel}^{-1}$, so a 528×528 patch subtends about $4.9'$ on a side. The labelled subset comprises 178 full-frame image-mask pairs: 8-bit PNG renders derived from FITS exposures, each paired with a hand-annotated binary trail mask (Figure 2.1). The original FITS files and their headers were unavailable, so the dataset does not include WCS, exposure, photometric-calibration, or observing-geometry metadata. Therefore, analysis of sky-coordinate projection, horizon or zenith angle, exposure-time-normalised trends, or calibrated flux behaviour is not attempted.

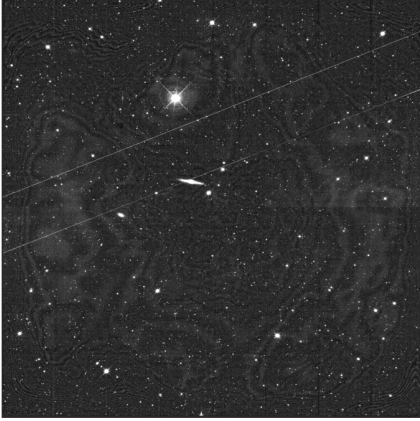
2.2 Patch extraction and class imbalance

Full frames are too large for direct training. Patches are extracted at 528×528 pixels with stride 528, yielding exactly 400 non-overlapping patches per image (20×20 grid). Many trails cross adjacent patches, and the natural patch distribution is highly imbalanced, with approximately 93–94 % of full-grid patches containing no trail pixels. To form the sampled patch manifests (patch lists used by the training and evaluation), all positive patches are retained and negatives are down-sampled to a 1:3 positive-to-negative patch ratio. Figure 2.2 gives the realised 528-pixel manifest sizes used by the final run.

The sampled test set is the primary test population since it has the same 1:3 pos:neg prior as the validation set used for threshold selection. A separate parity test set keeps every test-frame patch, including all negatives, to restore the natural empty-patch prevalence for comparison with the paper (Figure 2.2). This separation is crucial because adding many empty patches primarily changes precision: recall is computed over retained positive patches, whereas the additional negatives can introduce false positives.

Random Image and Mask Pairs

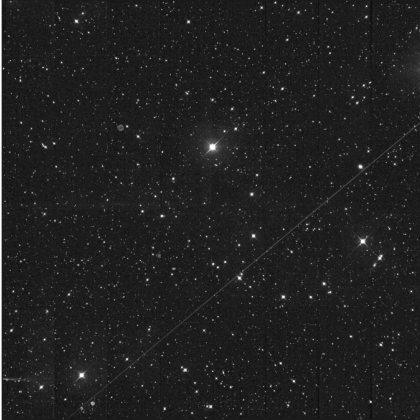
Image: ML1_20220728_180558_red.fits_full



Mask: ML1_20220728_180558_red_mask



Image: ML1_20191118_214959_red.fits_full



Mask: ML1_20191118_214959_red_mask

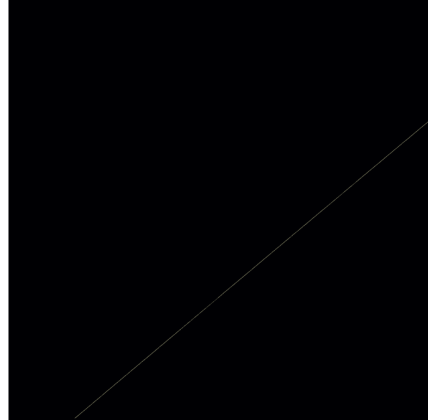


Figure 2.1: Example image–mask pairs from the labelled subset: 8-bit PNG renders with hand-annotated binary trail masks. Trails are long, near-linear features against a crowded stellar background.

2.3 Dataset splits

Figure 2.2 summarises the path from frames to evaluation manifests. Splits are performed at image level, never after patching, so every patch from a given frame remains in the same split. The 178 images have skewed trail-pixel counts and are stratified by total trail-pixel count into four quartile groups. The nominal 0.70/0.15/0.15 ratios are applied within each group with flooring; the remainder goes to test. The realised split is **122 train** / **24 validation** / **32 test** (0.685/0.135/0.180). Because splitting happens before patch extraction, this prevents patches from the same source image appearing in different splits and reserves a relatively large held-out fraction for evaluation.

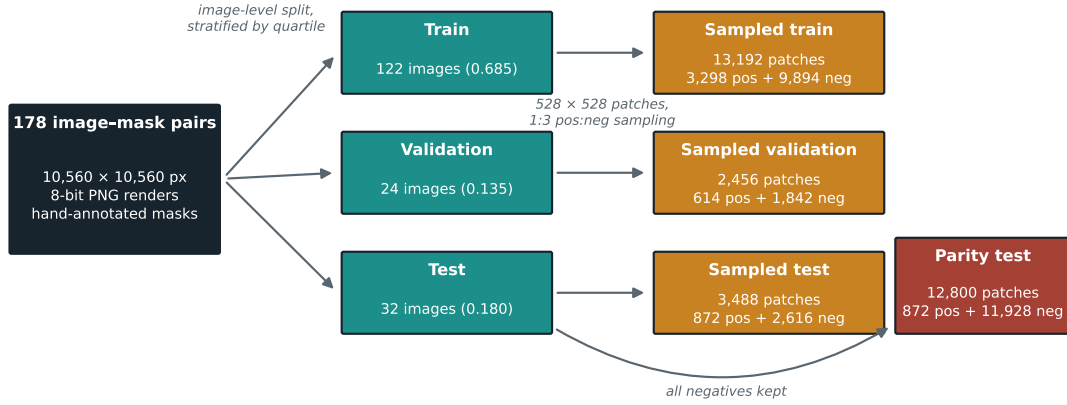


Figure 2.2: Data flow for the labelled frames. Sampled manifests drive training, threshold selection, and internal evaluation; the parity manifest restores all test-frame negatives for the paper comparison and is never used for selection.

2.4 Mask quality

Beyond visual inspection, connected-component measurements provide a simple mask-quality check. Components have high aspect ratios and a median stroke width near 6 px, estimated from distances to the nearest mask edge, matching long, thin annotated structures. The densest-patch overlays show masks following the visible trails, while exposing occasional endpoint under-coverage (Figure 2.3).

The masks are sufficient for supervised training, with one characterised caveat: full-frame overlays show occasional mask under-coverage at faint endpoints and low-contrast stretches. If the model detects a visible trail segment omitted from the mask, those pixels are still counted as false positives. Thus, label-referenced precision is a conservative estimate where masks omit visible trail pixels. Section 5.6.2 explores this sensitivity.

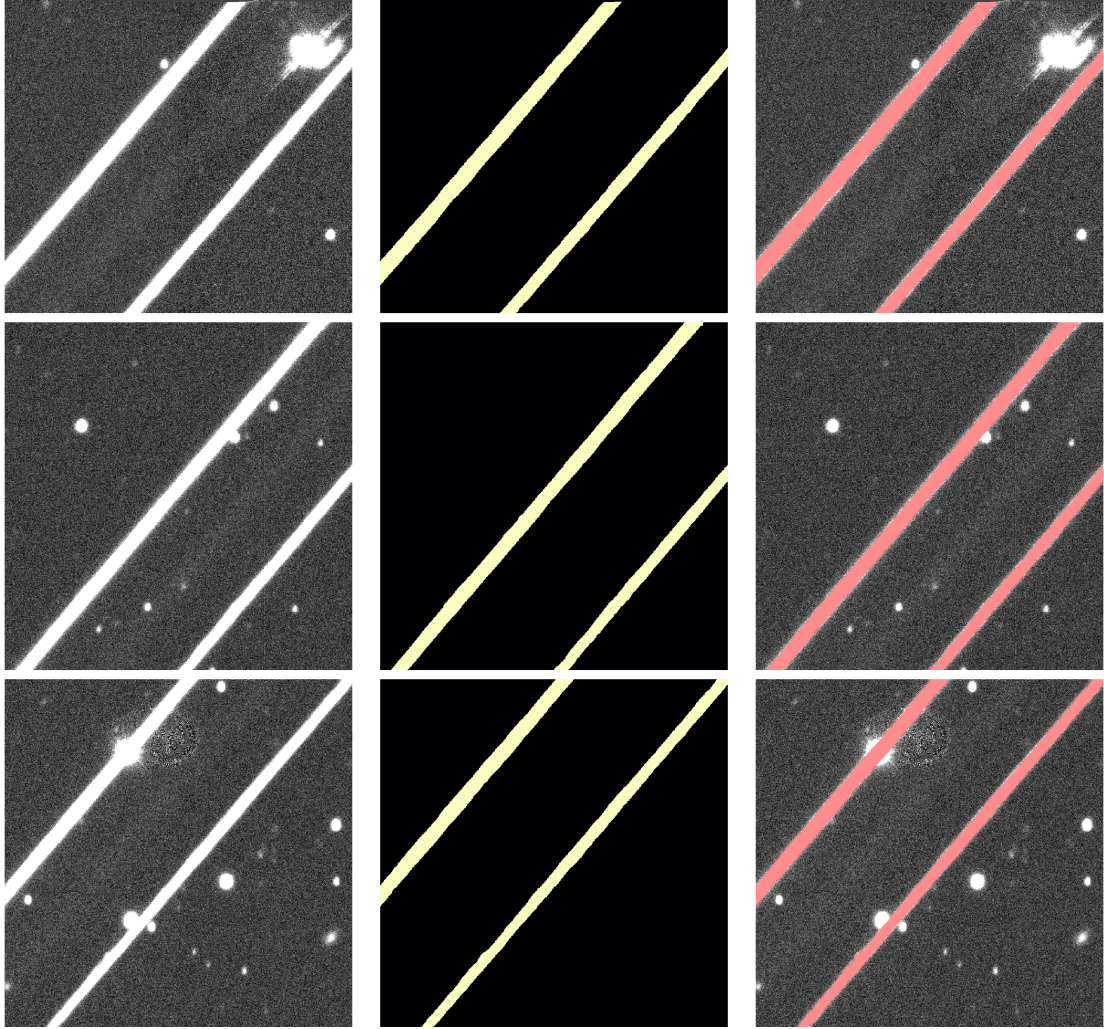


Figure 2.3: Densest-patch mask-quality examples: image, annotated mask, and overlay (red). Masks follow the visible trails, with occasional endpoint under-coverage.

Chapter 3

Methodological Framing

3.1 Problem formulation

Trail detection is posed as patch-wise binary pixel segmentation. Given a 528×528 single-channel patch, the model estimates a per-pixel probability that each pixel belongs to a satellite trail rather than background. Thus, the required output is a mask for downstream cleaning, not a catalogue of satellite identities or trail instances.

This is semantic segmentation: pixels are labelled by class, not assigned to individual trail instances. Crossing or near-parallel trails within a patch are simply all “trail”, so pixel-level metrics cannot distinguish merged from separated trails. The formulation is label-referenced: success is measured against the available binary masks, so boundary placement and mask under-coverage impact precision.

3.2 Segmentation with Hough post-processing

The pipeline pairs a learned segmentation stage with a classical geometric stage. A U-Net produces a soft-detection map that preserves weak, spatially extended evidence of a faint trail. A classical Hough detector applied directly to the raw image responds mainly to stars, galaxies, and diffraction structure. In a validation-tuned raw-image Hough implementation, precision was only about 10^{-3} , showing that this Hough voting is an insufficient masking baseline. Thus, the probabilistic Hough transform is used after the U-Net, where the input is already a trail-probability canvas rather than a crowded sky image. It can exploit the prior that a trail is globally linear to bridge gaps, recover continuous trail structure, and support high-recall masking. The methodological choices follow this: the U-Net supplies pixel-level segmentation, the BCE–Dice loss handles severe foreground imbalance, and evaluations separate selection from label-boundary effects.

3.3 U-Net architecture choice

The U-Net encoder–decoder with skip connections [20] preserves fine spatial detail by combining matching decoder and encoder features. This is important for thin, elongated, sparse targets, while its encoder path still supplies enough context to distinguish trails from stellar structure and background gradients. The same architecture underpins astronomical artefact detectors such as deepCR [33] and MaxiMask [19], supporting suitability here.

More elaborate variants are explored in Section 5.7.2, which tests whether adding attention changes the error pattern. Transformer-style segmenters such as ViT are not explored since they would change the detector family and need far more tuning and data to evaluate fairly on this small labelled subset.

3.4 Combined BCE and Dice loss

The foreground occupies a tiny fraction of pixels, so a pure cross-entropy objective is dominated by empty background. The Dice term measures region overlap and is less dominated by the background majority [15]. Combining it with binary cross-entropy retains a per-pixel classification term while following the Combo-loss pattern for severely imbalanced segmentation [26]. The final pipeline uses an unweighted BCE term; class imbalance is handled through 1:3 patch sampling and the Dice component, since additional positive weighting would favour over-prediction.

3.5 Evaluation and assumptions

Model and threshold selection are fixed in advance and validation-only, so no test metric is inspected before the configuration is fixed. Sampled and parity test metrics are kept distinct: the sampled set matches the validation prior, while the parity set restores the natural empty-patch prevalence used for paper comparison. Uncertainty treats the source image as the independent unit since patches from one frame share background, atmospheric image quality, and annotation structure. Finally, because masks omit some visible trail pixels (Section 2.4), pixel-overlap scores are complemented by topology-aware (clDice), false-positive distance, and boundary-tolerant metrics (Section 4.8).

Chapter 4

Model Implementation and Training

4.1 Data pipeline and augmentation

The patch builder implements the extraction scheme from Section 2.2 by writing 528×528 stride-528 patches at the 1:3 sampling ratio. Each manifest row records the source image, split, patch coordinates, positive-pixel fraction, and full-image mean and standard deviation. The last two fields implement the `full_image` normalisation mode: each patch is converted to $[0, 1]$ and normalised with its parent image statistics, matching the paper’s per-image standardisation rather than normalising each patch.

Training patches are augmented with random horizontal flips, vertical flips, and $k \times 90^\circ$ rotations applied identically to each image and mask (validation and test patches are not augmented). Furthermore, training images receive signal-dependent stochastic noise (masks and evaluation splits do not). This is not modelled as Poisson–Gaussian CCD noise since the inputs are display renders rather than calibrated electron counts. For a patch-pixel value $p \in [0, 1]$ the augmentation samples

$$p_{\text{noisy}} = \text{clip}\left(p + \mathcal{N}\left(0, m\sqrt{\alpha p + \beta^2}\right), 0, 1\right),$$

where m is the noise multiplier and α, β are calibrated once from empty-mask training patches, so the injected noise matches the measured background variation. The chosen calibration uses 9,894 empty train patches, giving $\alpha = 0.0209$ and $\beta = 0.0886$. A preliminary three-seed screen compared a no-noise control with multipliers $m \in \{0.5, 1.0, 2.0\}$, and the active configuration fixes $m = 1.0$ for robustness since validation- F_1 gain was not decisive.

4.2 Network architecture

The network shown in Figure 4.1 was reconstructed from the released Stoppa et al. Keras `model-best.h5` configuration. It is an encoder–decoder with skip connections, filters pro-

gressing $8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$, average-pooling downsampling, transposed-convolution upsampling, and LeakyReLU activations with slope 0.3. Each double-convolution block follows **Conv2d** $\rightarrow$ **LeakyReLU** $\rightarrow$ **Dropout** $\rightarrow$ **Conv2d** $\rightarrow$ **LeakyReLU**, with element-wise dropout in all nine blocks. The dropout rates are [0.1, 0.1, 0.2, 0.2, 0.3, 0.2, 0.2, 0.1, 0.1]. The network uses no batch normalisation. The 3×3 convolutions use He-normal initialisation [11]; the transposed convolutions and the final 1×1 head use Glorot-uniform initialisation with all biases initialised to zero. Inputs are 528×528 single-channel patches and the output is a single-channel logit map with identical spatial dimensions. Sigmoid conversion is applied outside the model in the loss, thresholding, Hough, and visualisation paths. Altogether, the network has 485,673 trainable parameters (`base_channels = 8`).

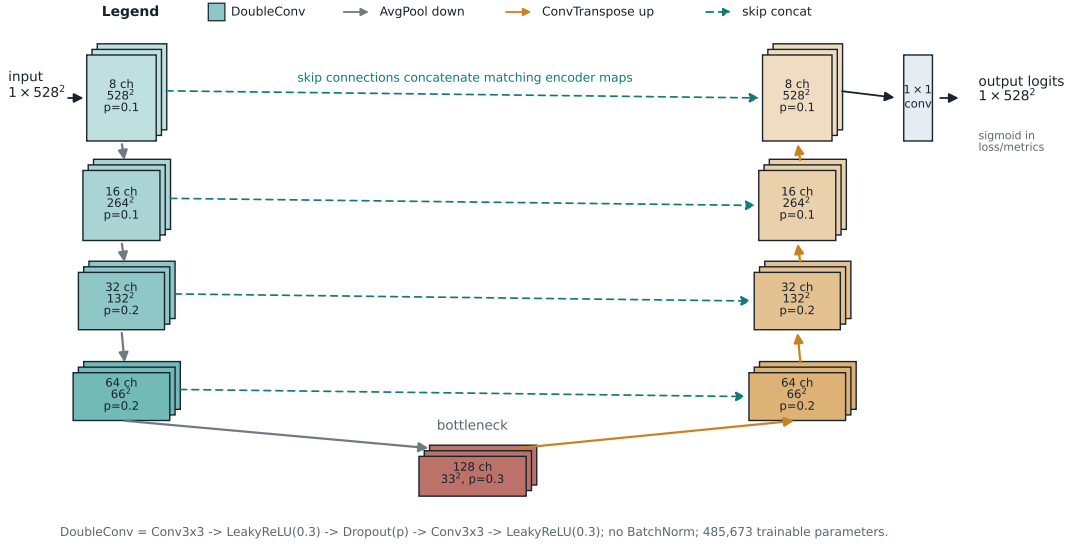


Figure 4.1: Architecture-faithful U-Net reconstructed from the released Keras model; labels give channel widths, spatial scale, and dropout.

4.3 Loss function

Training uses a combined binary cross-entropy (BCE) and squared-Dice loss:

$$\mathcal{L} = \lambda_{\text{BCE}} \mathcal{L}_{\text{BCE}} + \lambda_{\text{Dice}} \mathcal{L}_{\text{Dice}}, \quad \lambda_{\text{Dice}} = 1 - \lambda_{\text{BCE}}, \quad (4.1)$$

where

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_i p_i g_i + \epsilon}{\sum_i p_i^2 + \sum_i g_i^2 + \epsilon}, \quad \epsilon = 10^{-4},$$

with p_i denoting the sigmoid probability and g_i the binary target. The BCE term uses PyTorch’s BCE-with-logits loss without a class-balancing `pos_weight`, matching the paper and avoiding a second imbalance correction on top of the sampled patch prior. Losses are

evaluated in FP32 even though the forward pass uses BF16 autocast, avoiding low-precision accumulation in the BCE and Dice reductions.

4.4 Hyperparameter search

Hyperparameter search uses Optuna [1] with a tree-structured Parzen estimator and median pruner ($n_{\text{startup}} = 15$, 10 warm-up epochs). The 45 trials are equally balanced across batch sizes $\{8, 16, 32\}$. Within each batch-size, the continuous hyperparameters are learning rate and BCE weight λ_{BCE} ; dropout, normalisation, and noise multiplier are fixed. Appendix Table A.1 gives the full search-space and fixed-configuration table. Each trial runs for 20 epochs and is scored by threshold-swept validation F_1 , matching the final selection criterion. Pruning uses in-epoch validation Dice at the fixed training threshold as an early-pruning signal; only completed trials are rescored by swept validation F_1 . The top five trials by validation F_1 are retrained for 75 epochs at five seeds (2804, 1234, 42, 7, 13), with the winner chosen by mean threshold-swept validation F_1 across seeds *before any test metric is inspected*. All stochastic training runs set Python, NumPy, PyTorch, and DataLoader seeds; remaining training variability is reported through the five-seed restudy. The fixed tie-breaker, applied when means fall within 0.001 validation F_1 , prefers simpler/lower-risk configurations. The selected base U-Net is trial 44 (`batch_size=32`, learning rate 2.89×10^{-4} , $\lambda_{\text{BCE}} = 0.655$), with mean validation $F_1 = 0.854$ across the retrained seeds.

4.5 Threshold selection

A precision–recall curve is swept on the validation set to select the F_1 -optimal threshold, which is applied unchanged to the test evaluation. Each reported seed uses its own validation-selected threshold, with 0.45 for the primary seed 2804 (the paper fixes 0.58). Within each run, the saved best epoch is selected by validation Dice at the fixed training threshold. That checkpoint is then swept over thresholds on the validation set, and the resulting best validation F_1 is used for trial ranking and selecting the probability threshold used at test time.

4.6 Hough post-processing

A probabilistic Hough transform [14] (`cv2.HoughLinesP`) is applied to a lower-threshold probability canvas. It recovers faint, linear detections below the segmentation threshold. Its role is completeness, with precision assessed separately because of the line-drawing convention. For each test image the per-patch sigmoid probability maps are max-pooled into a canvas before Hough voting. The sampled evaluation uses the 1:3 patch support,

while the parity Hough comparison uses full 528 px tiling over every test image. A full-coverage tiled pass is used as an implementation check against patch-sampling artefacts. The canvas is thresholded at $t_{\text{Hough}} = 0.1$, after which `HoughLinesP` is run with a vote threshold of 50, a minimum line length of 100 px, and a maximum gap of 250 px, matching the implementation from Stoppa et al. [24]. Detected segments are drawn back onto the canvas at 3 px thickness, making line rendering a comparability caveat for precision.

4.7 Patch CNN classifier gate

A two-stage variant tests whether a patch-level gate can reduce empty-patch false positives and U-Net inference cost without losing trail-bearing patches. Section 5.7.1 explores placing a lightweight convolutional neural network patch classifier in front of the U-Net. The classifier is simple: a four-block strided encoder followed by global average pooling and a single fully connected logit head. It is trained on the sampled training manifest with labels derived from the patch positive-pixel fraction and a `pos_weight` equal to the train-split negative-to-positive ratio. Validation reports both the maximum- F_1 and a high-recall (≥ 0.99) operating point, because a patch rejected by the gate is unrecoverable downstream. The classifier and U-Net are trained independently and composed at inference, so the gate changes the pipeline without retraining the segmentation model.

4.8 Evaluation metrics

Segmentation quality is reported with pixel precision, recall, Dice (equal to F_1 for binary masks), and IoU. Since trails are thin connected structures, pixel overlap alone can miss whether a prediction preserves centreline continuity. Therefore, topology is assessed with the centreline Dice (clDice) [22]. Sensitivity to boundary placement is assessed with a boundary-tolerant precision/recall/ F_1 score at $k \in \{0, 1, 2, 3\}$ px, where $k = 0$ is the strict metric and $k > 0$ allows a predicted pixel and a ground-truth pixel to match within a small spatial tolerance. The spatial structure of false positives is summarised by their distance to the nearest ground-truth trail pixel, with whole-patch false positives reported separately as they have no defined trail distance. Uncertainty is quantified by a cluster bootstrap over source images, resampling images with replacement and aggregating their patches before recomputing metrics. Patch-level bootstrap is reported as a secondary check.

Chapter 5

Replication and Error Diagnosis

5.1 Replication of the target detector

The validation-selected winner (trial 44; Section 5.2) is evaluated on the held-out test images at its selected threshold of 0.45. Table 5.1 compares the sampled test set, the five-seed mean, and the parity test set. Recall remains close to the paper’s 0.94, which matters for high-recall trail masking. Precision remains lower, and falls further on the parity set when the natural empty-patch rate is restored. The rest of the chapter analyses this gap rather than treating it as an optimisation target.

Table 5.1: Replication comparison with Stoppa et al. (2024). Paper metrics use threshold 0.58; this work reports sampled and parity tests at validation-selected thresholds. Dashes indicate values not reported.

Run	Threshold	Precision	Recall	Dice	IoU
Stoppa et al. (2024)	0.58	0.94	0.94	–	–
This work, sampled (single seed)	0.45	0.852	0.918	0.884	0.791
This work, sampled (5-seed mean)	0.41–0.50	0.847	0.923	0.884	0.792
This work, parity (single seed)	0.45	0.780	0.918	0.843	0.729

5.2 Validation-only model selection

Trial 44 was selected by mean validation F_1 across the five-seed retraining of the top five trials, before any test metric was inspected (Section 4.4; Figure 5.3). Its validation precision–recall sweep sets the operating threshold of 0.45, carried unchanged into the seed-2804 test evaluation. Across the five seeds, the validation-selected thresholds range from 0.41 to 0.50, and each seed is evaluated at its own validation-selected threshold. Figure 5.2 shows the validation sweep for seed 2804; the F_1 plateau is flat between roughly 0.4 and 0.6, so small threshold shifts would not affect validation F_1 .

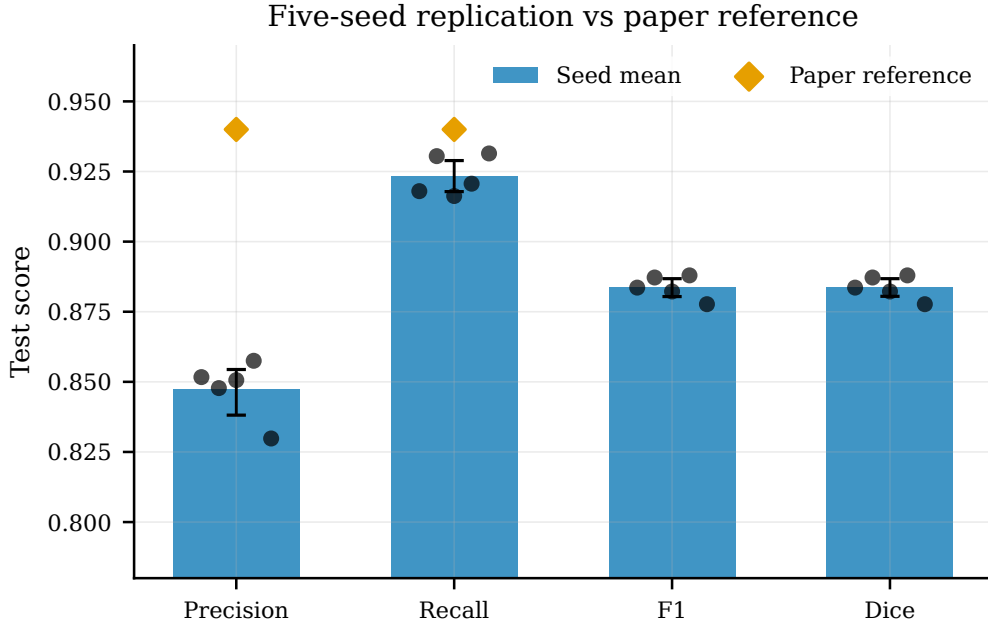


Figure 5.1: Five-seed replication summary for the validation-selected U-Net configuration. Paper-reference markers are for the explicitly reported precision and recall values.

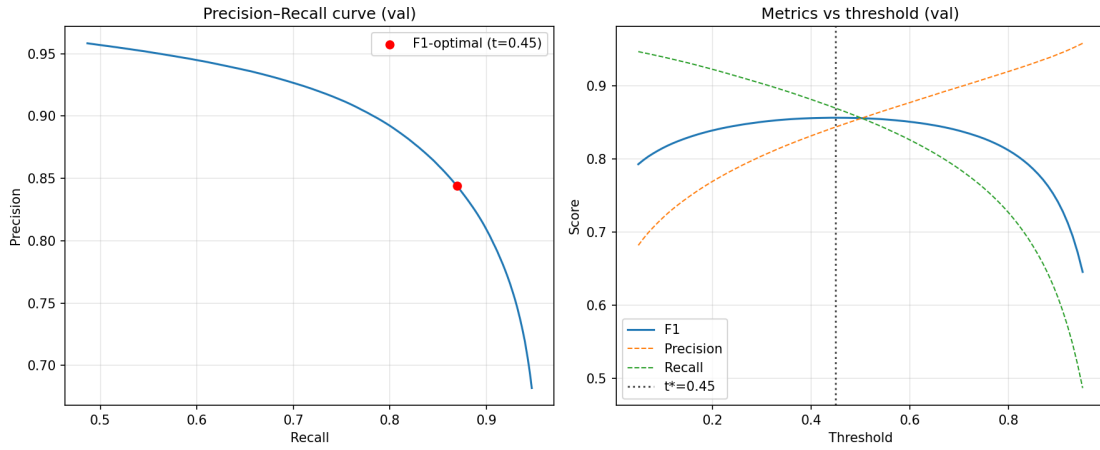


Figure 5.2: Validation-only threshold sweep for the primary seed-2804 winner. The vertical line marks $t^* = 0.45$; the paper's fixed 0.58 lies on the flat F_1 plateau.

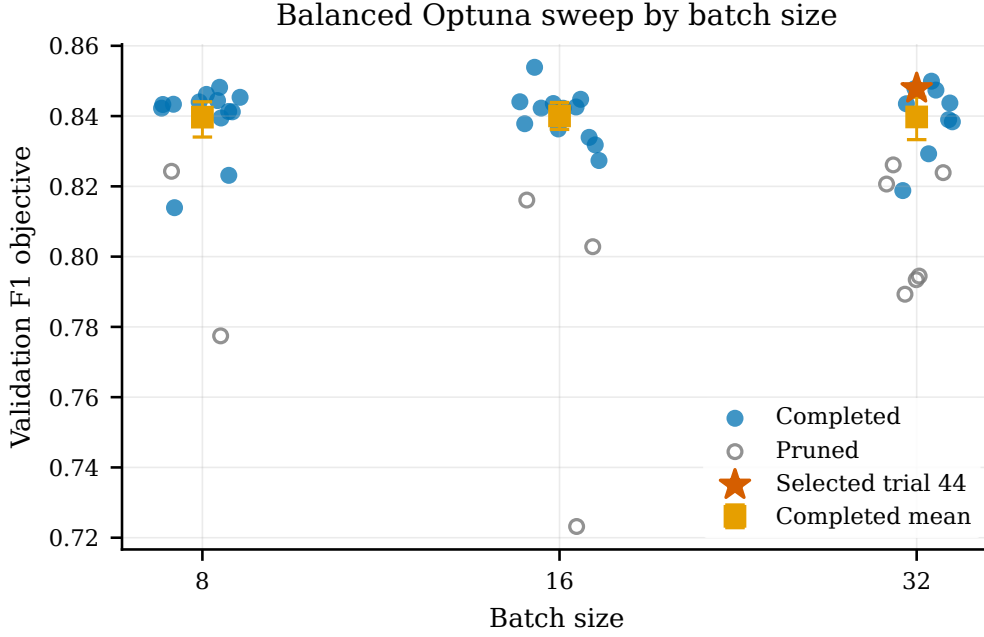


Figure 5.3: Balanced Optuna sweep by batch size; the star marks trial 44 after validation-only top-five retraining and tie-breaking.

5.3 Hough improves pixel completeness

Probabilistic Hough post-processing raises trail-pixel recall from 0.918 to 0.990 for the seed-2804 run on full-coverage parity canvas, equivalently reducing the pixel-level false-negative rate from 0.082 to 0.0105 (five-seed mean $0.077 \rightarrow 0.0098$). This missed-pixel reduction is comparable to the paper’s 6.98 % $\rightarrow$ 3.38 %. The full-coverage tiled pass described in Section 4.6 reproduced the parity canvas pixel-for-pixel, ruling out a patch-sampling or seam artefact. The image-level detection false-negative rate is degenerate here: every positive test image is already hit by at least one predicted pixel, so the rate is 0 both before and after Hough and is not comparable to the paper’s image-level rate. At patch level, the rate falls only from 0.122 to 0.103, since this stricter check counts trail-bearing patches with no recovered pixel. Thus, Hough is treated as a high-recall masking stage, with precision evaluated separately.

Figure 5.4 shows Hough in action: the U-Net misses a trail stretch over a bright star, and Hough recovers ground-truth pixels that appear only after the geometric stage.

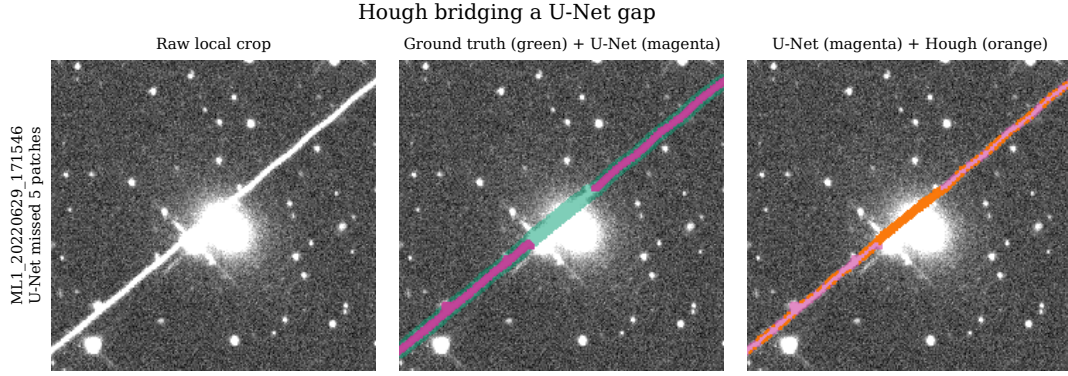


Figure 5.4: Hough bridging a U-Net gap: raw image, ground truth with U-Net prediction, and U-Net prediction with Hough bridge.

5.4 Uncertainty quantification

Using a cluster bootstrap over the 32 held-out test images (1,000 resamples, single-seed winner), the 95 % intervals are precision $[0.806, 0.891]$, recall $[0.888, 0.946]$, Dice $[0.857, 0.904]$, and IoU $[0.749, 0.825]$. These image-level intervals are several times wider than the patch-level bootstrap, represented by Dice $[0.876, 0.892]$. Patches from the same exposure share background, trail structure, and annotation structure, so the image-cluster interval is the headline interval. Across the five trial-44 seeds, variation is smaller but still visible: precision 0.847 ± 0.010 , recall 0.923 ± 0.007 , and Dice 0.884 ± 0.004 .

5.5 Qualitative prediction checks

Figure 5.5 contrasts low-Dice and high-Dice held-out patches for the selected U-Net. The low-Dice examples show typical failure modes around faint trails, bright-star structure, and partial boundary mismatch, while the high-Dice examples show clean trail recovery. Appendix A.3 adds empty-mask false positives and matched high-Dice true positives, with the same eight patches repeated for the attention variant.

The model was also applied to nine DECam frames with RECA-measured streaks (Figure 5.6). No metric is reported since ground-truth masks are not available for these frames. Manual visual review finds 8/9 visible streaks fully traced, with one faint streak partially traced and counted as a miss. This matches the low-contrast behaviour described in Section 5.6.4 and shows out-of-domain behaviour rather than a cross-dataset benchmark.

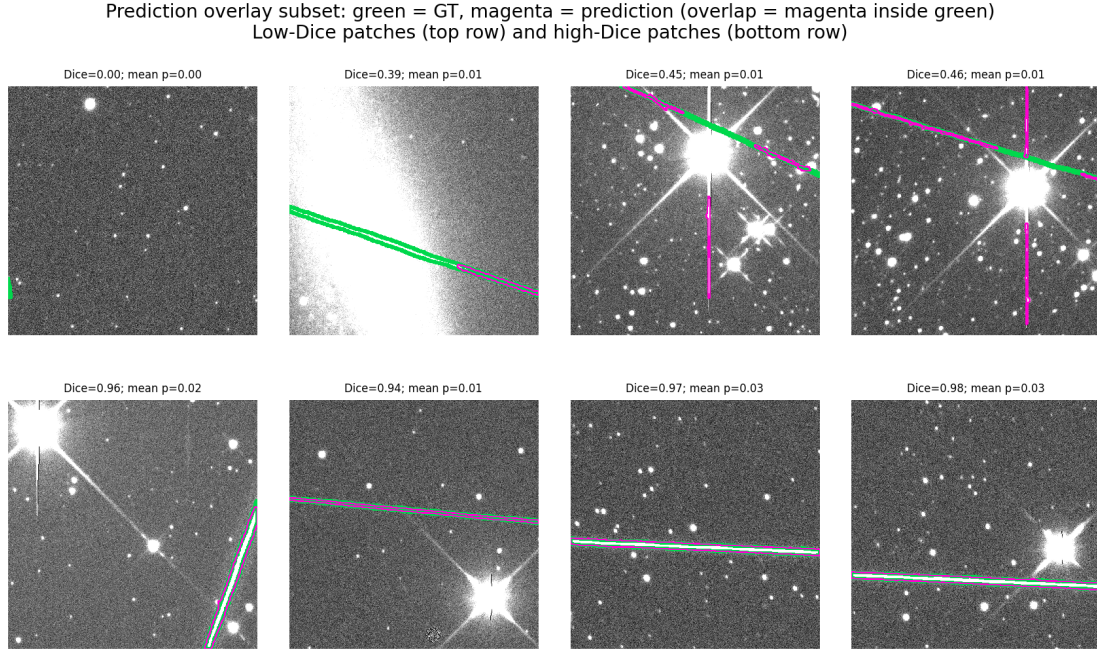


Figure 5.5: Low-Dice and high-Dice held-out test predictions for the selected U-Net: ground-truth mask in green and predicted mask in magenta.

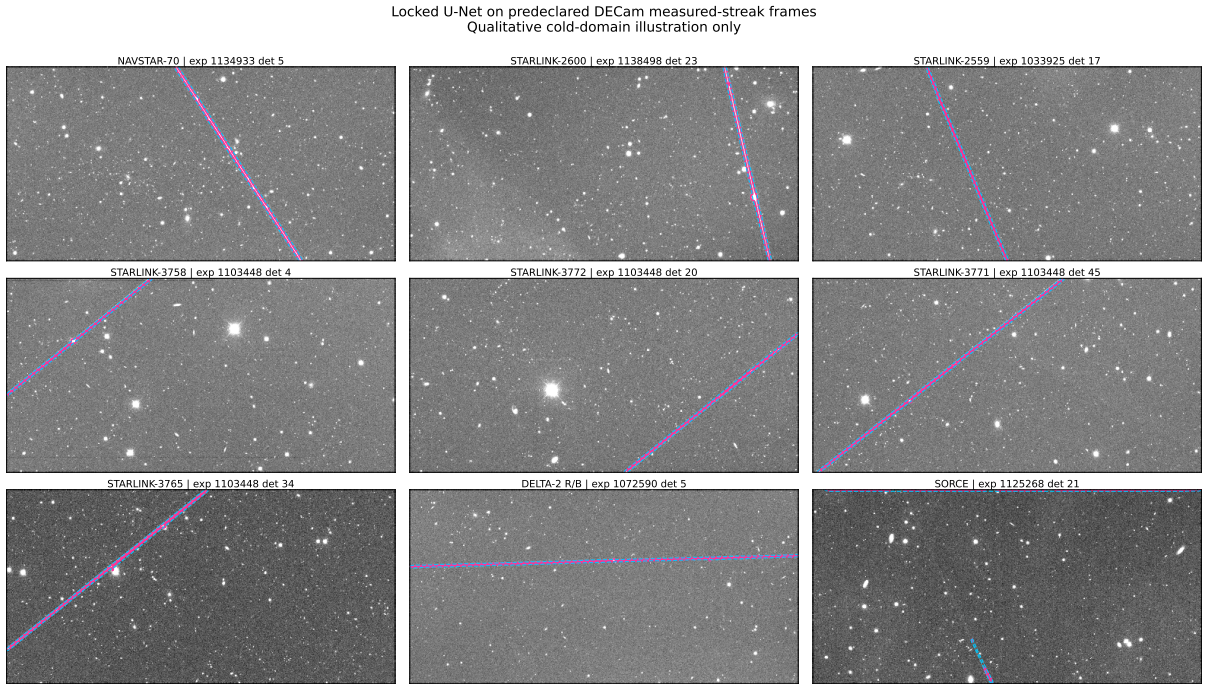


Figure 5.6: Qualitative DECam cold inference with the MeerLICHT-trained model: U-Net contour in magenta and Hough overlay in cyan. No ground-truth masks or benchmark metrics are available.

5.6 Precision gap near boundaries

The analyses below separate where the false positives occur from why they occur. Most false-positive mass is in trail-bearing patches, and the distance analysis shows that a majority of false-positive pixels lie within one pixel of labelled trails. This supports the boundary-scale interpretation, while the smaller inter-patch component consists of predictions in labelled trail-free patches which cannot be explained by mask-boundary placement. Section 5.6.6 tests this reading against a single-author re-annotation.

5.6.1 False-positive decomposition

Decomposing the false-positive pixels of the winning model shows that only 17.5 % are inter-patch (the network lighting up in trail-free patches), while 82.5 % are intra-patch (falling inside trail-bearing patches, though not necessarily adjacent to the trail) (Table 5.2). A patch-level gate can only remove the inter-patch share, so 17.5 % is the maximum false-positive mass it could remove. Hard-negative mining targets the same failure mode during retraining (Section 5.8.1), so this value is a rough scale for the possible effect.

Table 5.2: False-positive pixel decomposition. Only inter-patch false positives are directly removable by a patch-level gate.

Component	Pixels	Share
Intra-patch FP	300,745	82.5%
Inter-patch FP	63,662	17.5%

5.6.2 Boundary-tolerant evaluation

Re-scoring with symmetric boundary tolerances of $k \in \{0, 1, 2, 3\}$ px tests how much of the score is sensitive to boundary placement rather than larger misses (Figure 5.7). A predicted pixel and a ground-truth pixel are allowed to match if they fall within the chosen tolerance. Allowing a single pixel of tolerance, the five-seed mean precision rises from 0.847 (strict) to 0.946, recall from 0.923 to 0.980, and the ± 1 px F_1 reaches 0.962. The tolerant score is therefore diagnostic and should not be compared directly with the paper’s pixel score. It nevertheless shows that the precision gap to the paper largely closes under a one-pixel tolerance. A one-sided check separates this from a recall-for-precision trade: re-scoring the same predictions against ground truth dilated by one pixel raises precision from 0.852 to 0.937 but drops recall to 0.793, whereas the symmetric tolerance raises both. This supports the interpretation that much of the error is boundary-scale, with a smaller component far from any labelled trail. The tolerant score therefore only probes sensitivity: it relaxes the pixel-matching rule and is not directly comparable to the paper’s 0.94/0.94.

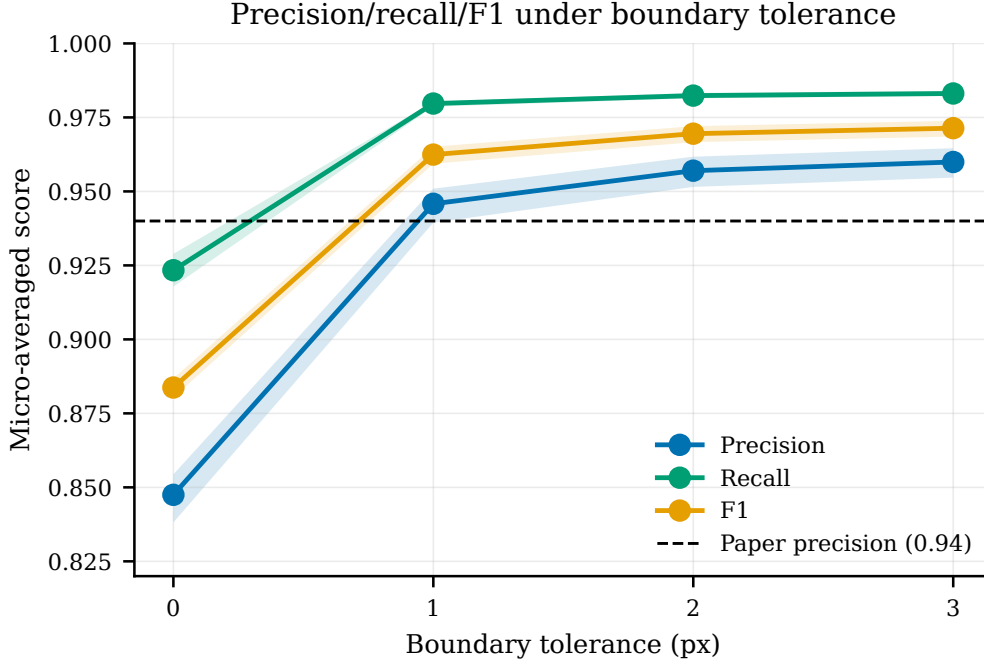


Figure 5.7: Boundary-tolerant metrics versus tolerance k ; the horizontal line marks the paper’s precision. The relaxed score is diagnostic and not a replacement for the paper’s pixel metric.

5.6.3 False-positive distance to mask

The distance from each false-positive pixel to the nearest ground-truth trail pixel concentrates at small values (Figure 5.8). Across all false positives, 57 % lie within 1 px of a labelled trail. Restricting the denominator to distance-defined false positives, which excludes the $\sim 17\%$ whole-patch component, raises this share to 69 %, with a median distance of 1.6 px. Both denominators are reported because the 69 % value excludes whole-patch false positives, so it would overstate the share of all false positives that are boundary-near. Together, these denominators support a boundary-near explanation for much of the precision gap, while leaving a smaller genuine-background component that boundary tolerance cannot explain.

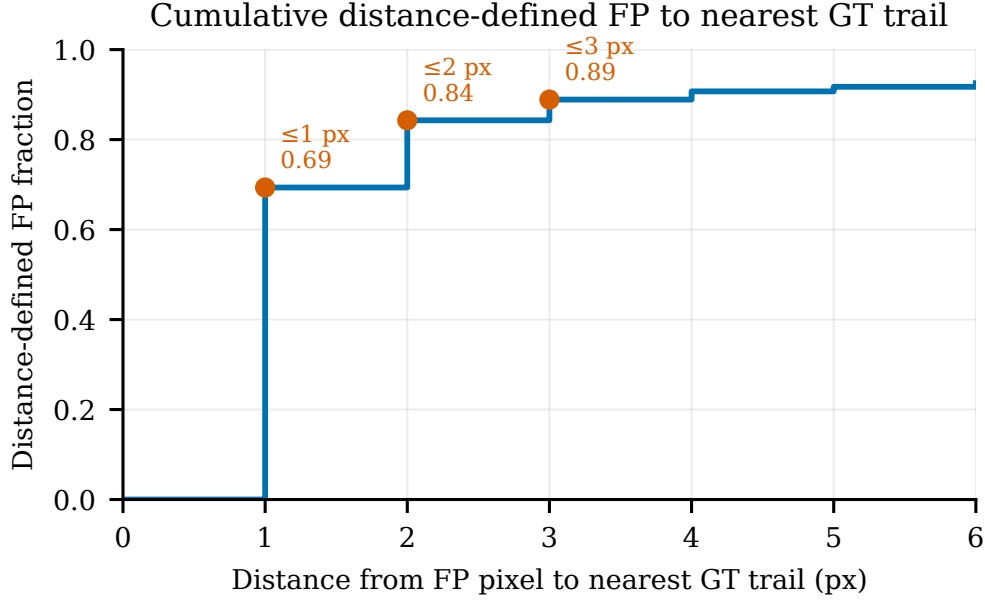


Figure 5.8: Cumulative distance of false-positive pixels to the nearest ground-truth trail, excluding whole-patch false positives; the prose reports both this denominator and all false positives.

5.6.4 Relative display-contrast checks

The winner was rescored post hoc on the 31 positive test images using raw 8-bit PNG intensities as a *relative display-contrast proxy* (Appendix Figures A.1–A.3). This is not calibrated flux or signal-to-noise, since the original count data are unavailable (Section 2.1).

Across 55 cleaned trail components split into three approximately equal relative-contrast bins, the lowest-contrast bin has the weakest pre-Hough recall (0.853) and the largest additional ground-truth recovery from Hough (0.092). After Hough, recall is at least 0.945 in all three bins.

Near-trail false positives have median display excess between the background and labelled-trail medians (Figure A.3). This is compatible with some boundary under-coverage, but remains descriptive rather than evidence of label error.

5.6.5 Topology-aware comparison (clDice)

As a topology check, clDice explores whether the predictions preserve the trail centreline structure, rather than only overlapping the labelled pixels. The centreline Dice tracks pixel Dice and preserves the architecture ordering: 0.847 for the base model versus 0.838 for the attention variant (-0.009), against pixel Dice 0.884 and 0.879 (-0.005). Thus, clDice gives the same null conclusion as pixel Dice: the attention model does not recover more continuous trail structure on this dataset.

5.6.6 Re-annotation audit

The prior analyses localise the error but leave two explanations: visible trails may be wider than their masks, or the model may systematically over-paint. To separate these, a single-author, evaluation-only re-annotation was performed on 64 sealed 528×528 crops. The set covered a range of display contrasts, including interior and endpoint trail crops for boundary scoring, plus false-positive-prone and quiet decoy crops as existence controls.

The first check is mask width. The original MeerLICHT masks, the re-annotation, and the model prediction all have median stroke width 6 px. Per-crop width differences are centred near zero: re-annotation minus original is +0.10 px (95% CI $[-0.30, +0.53]$), and model minus re-annotation is +0.22 px ($[-0.09, +0.54]$). These values do not support a systematic width bias, so the false-positive gap cannot be explained simply by labels being uniformly too thin or by the model being uniformly too wide.

The second check is pixel agreement. On the 44 interior and endpoint crops, the model matches the re-annotation ($F_1 = 0.893$) about as closely as the original labels do (0.890), while the model and original labels score 0.894 against each other (Table 5.3, pixel-weighted micro-averages). A paired per-crop bootstrap ($B = 10,000$, equal weight per crop) gives a model-minus-original difference against the re-annotation of -0.0001 (95% CI $[-0.024, +0.022]$). At ± 1 px the model is slightly lower ($\Delta = -0.015$, CI $[-0.038, +0.003]$), with the interval just spanning zero. Because the model was trained on the original labels, this is directional support rather than independent validation. The pattern is stable across groups, with median widths of 6 px and F_1 near 0.86 on interior segments and 0.89 on endpoints.

Table 5.3: Single-author blinded re-annotation audit over 44 interior and endpoint crops; values are micro-averaged F_1 scores between the original labels, re-annotation, and model.

Comparison	F_1	± 1 px F_1
Original labels vs re-annotation	0.890	0.978
Model vs re-annotation	0.893	0.970
Model vs original labels	0.894	0.972

The control crops test whether the model marks signal where the re-annotation does not. On the 11 false-positive-prone controls, the model detects signal in all cases, while the re-annotation marks a trail in only 5. On the six remaining crops, the median extra model area is 134 px, about 0.05 % of a crop. The quiet decoy crops produce no model detections. Figure A.6 shows overlays and Figure A.7 documents the annotation process. Since this audit had only one annotator, it measures disagreement between the original labels, the re-annotation, and the model. Its value is in testing whether a large width mismatch is visible; a multi-annotator audit remains the most valuable check (Section 7.2).

5.7 Model variants as error checks

The model variants below are diagnostic checks, with the selected U-Net retained as the baseline. If the precision gap were mainly caused by empty-patch false positives, a patch classifier should improve precision. If it reflected missing faint or continuous trail structure, attention gates might improve overlap or topology. The results are limited: the gate gives a bounded precision shift in line with the 17.5 % inter-patch false-positive share, and attention does not improve the base architecture.

5.7.1 Classifier-gated detector

The classifier-gated detector uses the CNN described in Section 4.7. Results are reported as a Pareto front over classifier and U-Net thresholds (Figure 5.9). Since the gate can only suppress empty-patch false positives, its possible precision gain is bounded by the inter-patch share.

At the illustrative high-recall operating point ($t_{\text{clf}} = 0.18$, $t_{\text{U-Net}} = 0.45$), the gate routes 2,625 patches to the U-Net (75.3 % of the sampled test set). Precision rises from 0.852 to 0.864, recall falls from 0.918 to 0.914, and Dice is 0.888. Classifier-rejected positive patches count as false negatives, so the classifier false-negative rate (0.096) sets a recall ceiling.

The gate therefore gives a bounded precision shift and a real but modest compute saving, skipping 24.7 % of sampled-test U-Net forwards. Larger savings would be expected only on full-grid evaluation, where empty patches dominate.

5.7.2 Attention-gated U-Net

The attention-gated U-Net [18] is a near-capacity-matched variant, carrying roughly 1 % more parameters than the base U-Net. It is trained under the same search and validation-only selection protocol as the base model (Section 4.4).

The selected attention model reaches test precision 0.838, recall 0.924, and Dice 0.879 at threshold 0.48, against the base Dice of 0.884. Across five seeds, the test means differ by at most 0.005 (Appendix Table A.2). The validation-restudy means are 0.854 for the base model (90 % interval [0.851, 0.856]) and 0.851 for the attention model ([0.850, 0.852]).

The result is therefore null under the matched protocol, not evidence for a decisive architectural ranking. Together with the clDice result (Section 5.6.5), it gives no evidence that attention recovers more continuous or faint trail structure on this dataset. Figure A.9 shows the attention-model gallery.

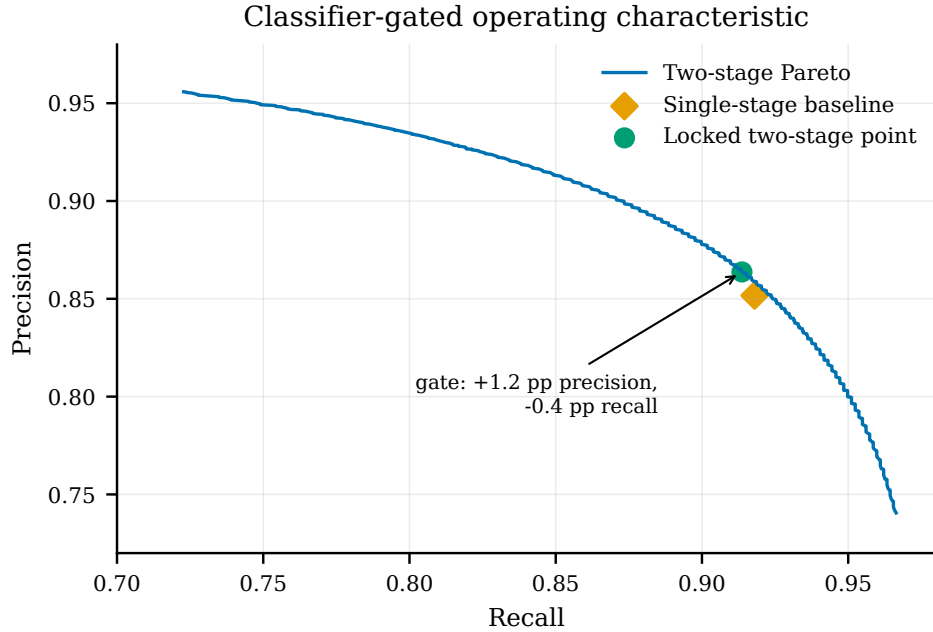


Figure 5.9: Two-stage operating-characteristic Pareto front over classifier and U-Net thresholds; the selected U-Net without the classifier remains the baseline.

5.8 Training and inference sensitivity

5.8.1 Data-side sensitivity

Two data-side checks ask whether more training images or targeted empty-patch examples materially change the precision gap.

Retraining the winning configuration on nested 30/50/70/100 % subsets of the training images, with validation and test sets fixed, gives an indicative learning curve (Figure 5.10). The curve is indicative because hyperparameters are transferred from the 100 % winner and each fraction is trained once. Validation Dice is flat to 50 %, rises at 70 %, and is then near-constant. Most of the gain appears before the final training fraction, suggesting that training-set size is unlikely to be the main limiting factor for this architecture.

Hard negatives were mined by running the model on labelled-empty patches and adding false-positive empty patches back into the training set. Adding 150 mined hard negatives gives a small precision gain in a retrain: $+0.007$ ($0.852 \rightarrow 0.859$), with essentially unchanged recall (-0.001) and $+0.003$ Dice. The precision shift is within the five-seed dispersion (± 0.010 ; Section 5.4), so the result is suggestive only. Its scale is consistent with the 17.5 % inter-patch false-positive share from Section 5.6.1.

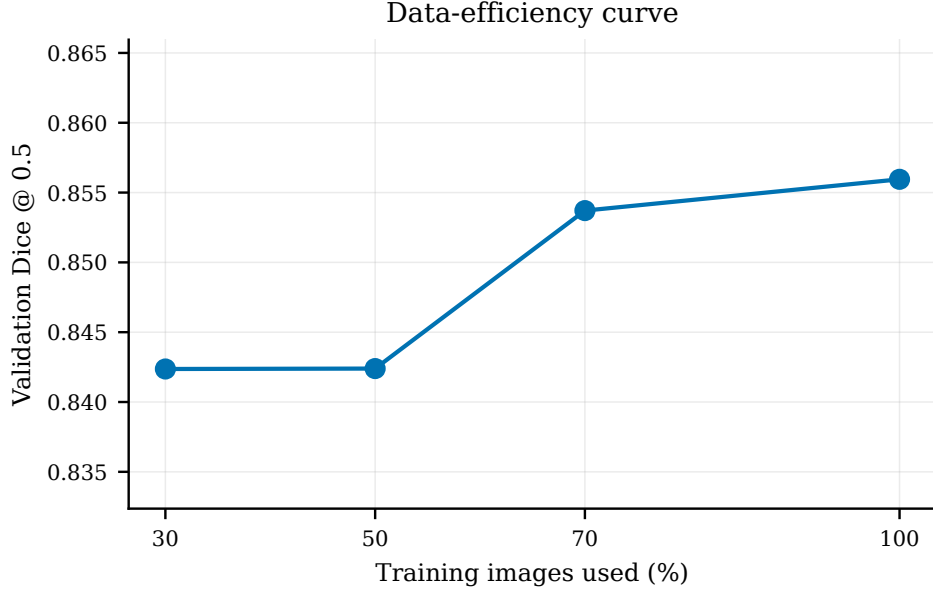


Figure 5.10: Validation Dice versus training-image fraction (fixed-threshold at 0.5). This is a single-seed indicative curve with hyperparameters transferred from the 100 % winner.

5.8.2 Soft labels and inference variants

These checks test whether the precision gap can be reduced by changing the training target or the inference procedure, while leaving the selected U-Net architecture intact.

The dilated soft-label pilot softens the target near trail boundaries. The trail core remains labelled 1, while a 1 px dilated boundary band receives an intermediate target value; validation and test masks remain hard-labelled. The main effect is wider predictions, with no localisation gain. Recall rises by +0.008, but precision falls by -0.022 ; exact Dice falls by -0.009 , ± 1 px F_1 is essentially unchanged (-0.001), and cIDice falls by -0.013 . Simply widening the training target therefore does not close the boundary-sensitive residual.

Two inference-time variants give similarly small changes. A five-checkpoint ensemble with test-time augmentation reaches Dice 0.8885 (precision 0.852, recall 0.929 at threshold 0.44), compared with 0.8836 for the selected single model. The +0.005 Dice change is comparable to five-seed dispersion and adds inference cost. Replacing the binary Hough input with a probability-stratified canvas changes pixel recall by only +0.00003, leaves the patch-level false-negative rate unchanged, and raises Hough false-positive pixels from 2,725,702 to 2,785,274. The binary Hough stage therefore appears recall-saturated on this dataset.

Chapter 6

Discussion

6.1 Differences from the target paper

Table 6.1 summarises the methodological and implementation differences that affect interpretation of the replication. Some follow from missing details in Stoppa et al. [24]; others are deliberate safeguards, including image-level splitting, validation-only threshold selection, and sampled-plus-parity testing.

Table 6.1: Documented methodological and implementation differences from Stoppa et al. (2024).

Aspect	This work	Target paper
Framework	PyTorch	Keras / TensorFlow
Architecture source	Reconstructed from the released Keras model, reimplemented in PyTorch	Released Keras model
Output convention	Logits; sigmoid in loss/eval	Built-in final sigmoid
Dice formulation	Squared-denominator (released ASTA code)	Standard Dice (paper text)
Patch tiling	528^2 at stride 528 (20×20 grid)	528^2 ; tiling unspecified
Threshold	Validation PR sweep (F_1 -optimal)	Fixed 0.58
Splits	Image-level, trail-pixel-stratified; 122/24/32	Not specified
Test sampling	Sampled 1:3 plus parity (all negatives)	20,000-patch set
Hough input	Patch-reconstructed; checked against full-coverage parity	Dense full image
Hyperparameters	Validation-only Optuna search	Not published
Noise augmentation	Calibrated train-only signal-dependent noise	Not specified

The swept threshold and different test population mainly affect precision by making it population-dependent, leaving recall as the closest like-for-like quantity (Section 5.1). The Hough comparison is checked on a full-coverage canvas, although precision remains sensitive to the 3px line-drawing convention (Section 5.3). The remaining differences are treated as part of the replication context rather than isolated one at a time: architecture tests retain the PyTorch reimplementation and output convention, image-level splitting removes patch-level leakage, and noise augmentation was adopted only after validation screening.

6.2 Interpreting the headline comparison

The differences in Section 6.1 make recall the closest like-for-like quantity. Even this comparison spans different populations: the 32 test images used here and the paper’s 20,000 patches with unreported class balance. For precision, the class-balance effect is measured directly. At identical recall and fixed threshold, precision falls from 0.852 to 0.780. Section 5.6 then places much of the remaining error near annotated boundaries, supporting boundary-scale annotation disagreement as the leading interpretation. Evaluating at the paper’s fixed 0.58 instead of the swept 0.45 raises precision by only 0.025, so threshold choice is a minor contributor.

The Hough stage improves pixel completeness (Section 5.3) but lowers pixel precision from 0.780 to 0.410. A line-thickness sweep attributes about half of this drop to the 3px line-drawing convention (Figure A.4). On the full-coverage tiled canvas, verified pixel-for-pixel against the parity canvas, the result is therefore evidence of comparable completeness. The tolerant scores carry the same caution: the ± 1 px precision 0.946 shows that much of the gap is sensitive to one-pixel boundary placement, and it is kept separate from the paper’s 0.94/0.94. The same pattern holds across the five seeds (Figure A.5).

6.3 Limitations

Six limitations define the scope of these findings.

- **Mask under-coverage.** The annotations occasionally omit faint endpoints and low-contrast stretches (Section 2.4). The effect is asymmetric: a correct detection of an unlabelled trail pixel scores as a false positive, and a model trained on such labels may also learn to suppress faint extensions. Label-referenced precision can therefore understate detector performance where masks omit visible trail pixels. The re-annotation (Section 5.6.6) finds no systematic width difference between the original labels and the re-annotation; a multi-annotator audit would better quantify any mask under-coverage.

- **Dataset size.** The subset holds 178 images, of which 32 form the test set. Image-level bootstrap intervals are therefore several times wider than patch-level ones (Section 5.4), the image-level detection rate has little room to separate models, and the protocol-variation checks each use a single seed.
- **Sampled evaluation distribution.** The sampled test set keeps a 1:3 positive-to-negative patch prior, which inflates precision relative to the full-image patch population, where roughly 93 % of patches are empty. The parity evaluation restores the negatives while retaining the sampled-validation threshold. The sampled-to-parity precision drop (0.852 to 0.780) measures the resulting shift; recall is unaffected because all positive patches are retained.
- **Hough line-drawing convention.** The parity Hough canvas was verified as a full-coverage tiled prediction, ruling out patch sampling as the source of this sensitivity. The remaining sensitivity comes from the 3 px line-drawing convention.
- **Single telescope.** All training and quantitative evaluation are confined to MeerLICHT, leaving transfer across instruments with different pixel scales, point-spread functions, or backgrounds untested. The DECam montage (Figure 5.6) therefore provides a qualitative transfer check.
- **PNG-only inputs.** The 8-bit PNG renders cap the dynamic range and remove WCS, exposure, and photometric metadata. Contrast statements in Section 5.6.4 are therefore measurements on the rendered PNG scale. The direction of any bias is unclear, because the labels were drawn on the same renders. The analysis is also single-channel, leaving any effect of multi-band imagery on trail separability or morphology untested [32].

6.4 Scientific implications

Beyond the replication, the results matter for astronomical survey practice because they separate detector utility from label-referenced scoring. A detector may recover enough trail extent to support masking even when pixel precision is reduced by boundary-level annotation choices. The analysis therefore has three implications for reporting, evaluation, and practical trail masking.

First, where masks omit visible trail pixels, label-referenced precision can be conservative. The Hough stage illustrates this: it extrapolates linear extent the network under-detects, and any recovered pixels outside the annotation score as false positives. The metric can therefore penalise the recovery the stage was built for. Reported precision for this detector family should be read alongside a statement of label quality; comparisons

across papers with different annotation rules can otherwise mix detector performance with labelling policy.

Second, pixel-level false-negative rates have an instance-level blind spot: the pixel-wise segmentation formulation labels every trail pixel identically, so merging two near-parallel trails into one mask can leave the pixel metrics used here unchanged. This matters for newly launched satellites that appear as several close, near-parallel trails: such masks can protect the image while still giving unreliable trail counts.

Third, the boundary-placement finding suggests a practical reporting standard. A majority of false-positive pixels lie within about one pixel of the labelled boundary, and no tested change resolves that gap. The re-annotation (Section 5.6.6) supports the interpretation that part of the score is limited by annotation granularity. Evaluations of thin-artefact detectors should therefore record a mask-width convention and report a ± 1 px boundary-tolerant operating point alongside the pixel score, so that boundary convention and detection failure can be interpreted separately.

For the motivating problem of satellite trails contaminating survey images, the work supports high-recall trail masking and identifies which errors matter for downstream use. Applying a one-pixel dilation before image stacking is a plausible downstream masking test because it targets the measured boundary error; its effect on stacking, photometry, and false masking remains untested here. The classifier gate is best treated as a compute feature: it skips 24.7% of sampled-test U-Net forwards, and likely more under the full-image patch distribution, while its precision gain is limited by the 17.5% inter-patch false-positive share. Because operating points depend on the image distribution, an operational detector should re-select its threshold on deployment-like data.

Chapter 7

Conclusions and Future Work

7.1 Conclusions

This dissertation replicated the Stoppa et al. [24] U-Net plus probabilistic Hough satellite-trail detector on a 178-image MeerLICHT subset. The architecture was reconstructed from the released Keras model, and both the model and threshold were fixed using validation data alone. The most stable comparisons with the paper are recall and Hough completeness: five-seed sampled-test recall is 0.923 ± 0.007 against 0.94, and trail-pixel recall rises from 0.918 to 0.990 on the full-coverage parity canvas after Hough. Pixel precision remains below the published level (0.847 sampled, 0.780 at parity) and depends on the test-set class balance, leaving recall as the like-for-like comparison across test populations.

The follow-up analyses make the precision gap interpretable: a one-pixel boundary tolerance raises five-seed precision from 0.847 to 0.946, and 57 % of false positives lie within one pixel of a labelled trail. A single-author re-annotation probes the main ambiguity. The original labels, the re-annotation, and the model all have median width 6 px, with no systematic width bias. Under this design, model-re-annotation agreement is not statistically distinguishable from original-label-re-annotation agreement. The tested interventions have small or null effects in the tested settings: classifier gate, attention U-Net, data efficiency, hard-negative mining, soft labels, ensembling, and probability-stratified Hough. These results make simple capacity or experimental-design explanations less likely without proving the boundary interpretation. None closes the gap materially, so boundary-scale annotation disagreement is the best-supported, but not proven, reading.

The broader methodological lesson is that faithful replication can diagnose discrepancies as well as confirm headline results. Here, validation-only selection and separate sampled/parity evaluations made the precision shortfall interpretable. For astronomy, the useful outcome is a controlled case showing that high-recall satellite-trail masking can coexist with boundary-limited pixel precision. Evaluations of thin artefact detectors should therefore report pixel scores with boundary-tolerant scores and mask-width conventions.

7.2 Future work

Future work should separate three questions: how much annotators agree on trail width, whether the detector transfers across instruments, and how masks affect downstream image products. The strongest first step is a *multi-annotator audit*. Independent annotators would establish inter-annotator agreement and turn the single-author audit of Section 5.6.6 into a stronger estimate of how much mask granularity contributes to the precision gap. A smaller mask-cleaning experiment would test component-level choices, such as removing isolated small objects before training and evaluation. The cleaned masks would define a new target and leave the reported results unchanged.

A second priority is a *labelled cross-instrument evaluation*, since the DECam application here is qualitative only (Figure 5.6). This evaluation should include a classical baseline following Serrano Mendoza et al. [21]. Attention [18] and richer architectures could then be re-examined, since the MeerLICHT matched-capacity null result may not generalise across instruments.

Further work should also address the limits of pixel-level scoring. An object-level trail-counting metric would expose the merged near-parallel-trail failure mode that pixel rates cannot detect. This could be made quantitative with a synthetic test set containing close, parallel satellite trails. Trail *removal* is one natural downstream step: the detected mask and fitted Hough geometry could be combined to interpolate across the trail. On 8-bit renders, however, this would test visual interpolation rather than photometric recovery. Together, these extensions would move the work from controlled replication and error diagnosis toward practical satellite-trail mitigation.

Bibliography

- [1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- [2] C. G. Bassa, O. R. Hainaut, and D. Galadí-Enríquez. Analytical simulations of the effect of satellite constellations on optical and near-infrared observations. *Astronomy & Astrophysics*, 657:A75, 2022. doi: 10.1051/0004-6361/202142101. arXiv:2108.12335.
- [3] D. Bektešević and D. Vinković. Linear feature detection algorithm for astronomical surveys – I. Algorithm description. *Monthly Notices of the Royal Astronomical Society*, 471(3):2626–2641, 2017. doi: 10.1093/mnras/stx1565.
- [4] S. Bloemen, P. Groot, P. Woudt, et al. MeerLICHT and BlackGEM: custom-built telescopes to detect faint optical transients. In *Ground-based and Airborne Telescopes VI, Proceedings of the SPIE*, volume 9906, page 990664, 2016. doi: 10.1117/12.2232522.
- [5] J. Carrillo Navarro et al. StreakMind: AI detection and analysis of satellite streaks in astronomical images with automated database integration. *Astronomy & Astrophysics*, 708:A211, 2026. doi: 10.1051/0004-6361/202558754. arXiv:2605.03429.
- [6] CelesTrak (T. S. Kelso). SATCAT and active-satellite catalogue. <https://celestrak.org/satcat/>, 2026. Active-satellite element table parsed 2026-05-27.
- [7] X. Chen, H. Gu, C. Wang, H. Mu, J. Zheng, J. Du, J. Ren, Z. Fan, and J. Li. Artificial satellite trails detection using U-Net deep neural network and line segment detector algorithm. *Publications of the Astronomical Society of the Pacific*, 137(9):094504, 2025. doi: 10.1088/1538-3873/ae051f. arXiv:2509.16771.
- [8] R. O. Duda and P. E. Hart. Use of the Hough transformation to detect lines and curves in pictures. *Communications of the ACM*, 15(1):11–15, 1972. doi: 10.1145/361237.361242.
- [9] Federal Communications Commission, Space Bureau. Order DA 26-36: SpaceX Gen2 Starlink authorisation. <https://docs.fcc.gov/public/attachments/DA-26-36A1.pdf>, 2026.

- [10] O. R. Hainaut and A. P. Williams. Impact of satellite constellations on astronomical observations with ESO telescopes in the visible and infrared domains. *Astronomy & Astrophysics*, 636:A121, 2020. doi: 10.1051/0004-6361/202037501. arXiv:2003.01992.
- [11] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015. doi: 10.1109/ICCV.2015.123.
- [12] P. V. C. Hough. Method and means for recognising complex patterns. *US Patent 3069654*, 1962.
- [13] S. Kruk et al. The impact of satellite trails on Hubble Space Telescope observations. *Nature Astronomy*, 7:262–268, 2023. doi: 10.1038/s41550-023-01903-3.
- [14] J. Matas, C. Galambos, and J. Kittler. Robust detection of lines using the progressive probabilistic Hough transform. *Computer Vision and Image Understanding*, 78(1): 119–137, 2000. doi: 10.1006/cviu.1999.0831.
- [15] F. Milletari, N. Navab, and S.-A. Ahmadi. V-Net: Fully convolutional neural networks for volumetric medical image segmentation. In *Fourth International Conference on 3D Vision (3DV)*, pages 565–571, 2016. doi: 10.1109/3DV.2016.79.
- [16] P. Mróz et al. Impact of the SpaceX Starlink satellites on the Zwicky Transient Facility survey observations. *The Astrophysical Journal Letters*, 924(2):L30, 2022. doi: 10.3847/2041-8213/ac470a. arXiv:2201.05343.
- [17] G. Nir, B. Zackay, and E. O. Ofek. Optimal and efficient streak detection in astronomical images. *The Astronomical Journal*, 156(5):229, 2018. doi: 10.3847/1538-3881/aaddff. arXiv:1806.04204.
- [18] O. Oktay et al. Attention U-Net: Learning where to look for the pancreas. In *Medical Imaging with Deep Learning (MIDL)*, 2018. arXiv:1804.03999.
- [19] M. Paillassa, E. Bertin, and H. Bouy. MaxiMask and MaxiTrack: two new tools for identifying contaminants in astronomical images using convolutional neural networks. *Astronomy & Astrophysics*, 634:A48, 2020. doi: 10.1051/0004-6361/201936345. arXiv:1907.08298.
- [20] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015. doi: 10.1007/978-3-319-24574-4_28.

- [21] A. Serrano Mendoza, M. Rawls, and A. A. Plazas Malagón. Identifying and measuring satellite streaks in DECam images. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2603.10790. arXiv:2603.10790.
- [22] S. Shit, J. C. Paetzold, A. Sekuboyina, I. Ezhov, A. Unger, A. Zhylka, J. P. W. Pluim, U. Bauer, and B. H. Menze. clDice – a novel topology-preserving loss function for tubular structure segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. arXiv:2003.07311.
- [23] D. V. Stark, N. A. Grogin, J. E. Ryon, and R. A. Lucas. Improved identification of satellite trails in ACS/WFC imaging using a modified Radon transform. *arXiv e-prints*, 2026. arXiv:2602.17816.
- [24] F. Stoppa et al. Automated detection of satellite trails in ground-based observations using U-Net and Hough transform. *Astronomy & Astrophysics*, 692:A199, 2024. doi: 10.1051/0004-6361/202451663. arXiv:2407.19461.
- [25] A. J. Storkey, N. C. Hambly, C. K. I. Williams, and R. G. Mann. Cleaning sky survey databases using Hough transform and renewal string approaches. *Monthly Notices of the Royal Astronomical Society*, 347(1):36–51, 2004. doi: 10.1111/j.1365-2966.2004.07211.x.
- [26] S. A. Taghanaki, Y. Zheng, S. K. Zhou, et al. Combo loss: handling input and output imbalance in multi-organ segmentation. *Computerized Medical Imaging and Graphics*, 75:24–33, 2019. doi: 10.1016/j.compmedimag.2019.04.005. arXiv:1805.02798.
- [27] J. A. Tyson et al. Mitigation of LEO satellite brightness and trail effects on the Rubin Observatory LSST. *The Astronomical Journal*, 160:226, 2020. doi: 10.3847/1538-3881/abba3e. arXiv:2006.12417.
- [28] Vera C. Rubin Observatory. Frequently asked questions: impacts from artificial satellites and debris. <https://rubinobservatory.org/for-scientists/frequently-asked-questions/leo-sats>, 2026. Accessed 2026-05-27.
- [29] C. Walker, J. Hall, et al. SATCON1 report: impact of satellite constellations on optical astronomy and recommendations toward mitigations. Bulletin of the AAS, 52(2); NSF NOIRLab / AAS, 2020.
- [30] C. Walker et al. SATCON2 working group reports. Bulletin of the AAS; NSF NOIRLab / AAS, 2021.
- [31] C. Xu, C. McCully, B. Dong, D. A. Howell, and P. Sen. Cosmic-CoNN: A cosmic-ray detection deep-learning framework, data set, and toolkit. *The Astrophysical Journal*, 942(2):73, 2023. doi: 10.3847/1538-4357/ac9d91. arXiv:2106.14922.

- [32] H.-J. Yu, J.-L. Zheng, and Y. Fang. Using deep learning to identify artificial satellite trails in multi-band photometric astronomical images. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2509.04081. arXiv:2509.04081.
- [33] K. Zhang and J. S. Bloom. deepCR: Cosmic ray rejection with deep learning. *The Astrophysical Journal*, 889(1):24, 2020. doi: 10.3847/1538-4357/ab3fa6. arXiv:1907.09500.

Appendix A

Supplementary Results

A.1 Post-hoc Trail and Error Checks

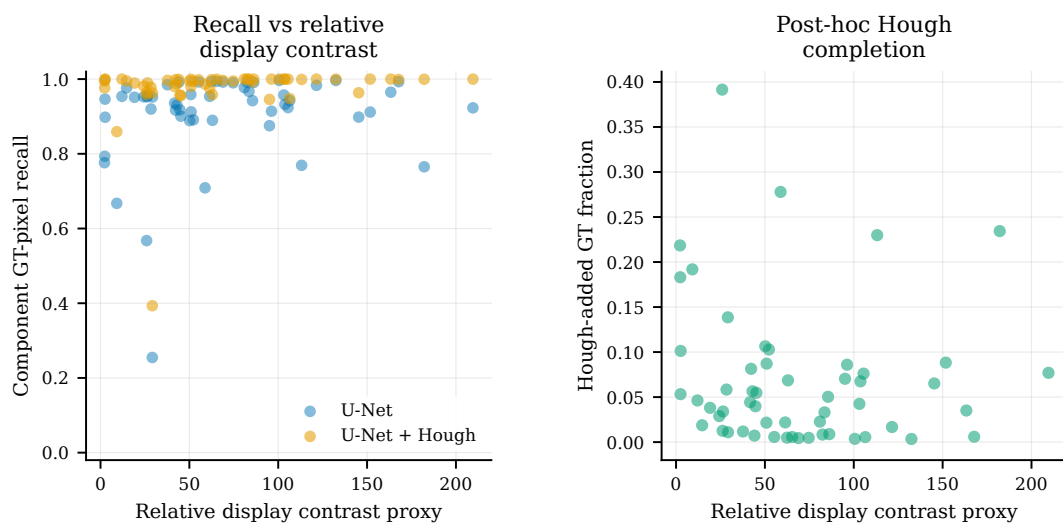


Figure A.1: Relative display contrast versus U-Net recall and Hough completion for the selected winner; the contrast proxy is descriptive, not calibrated flux or SNR.

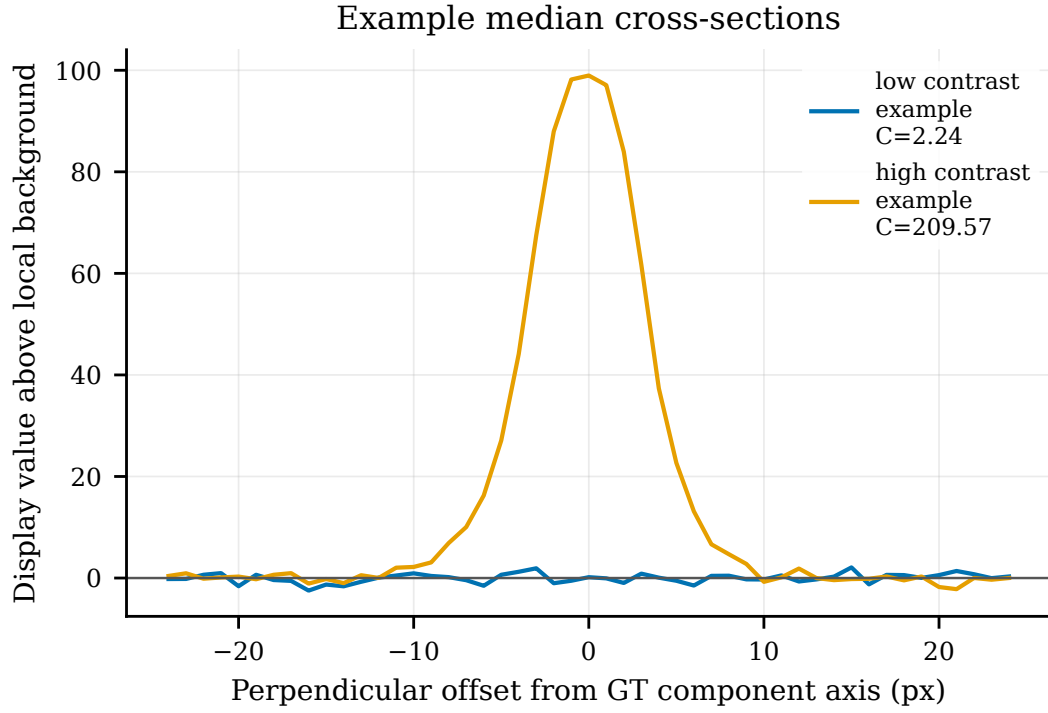


Figure A.2: Median cross-sections through labelled trails in display-space PNG values; descriptive examples of the relative-contrast proxy.

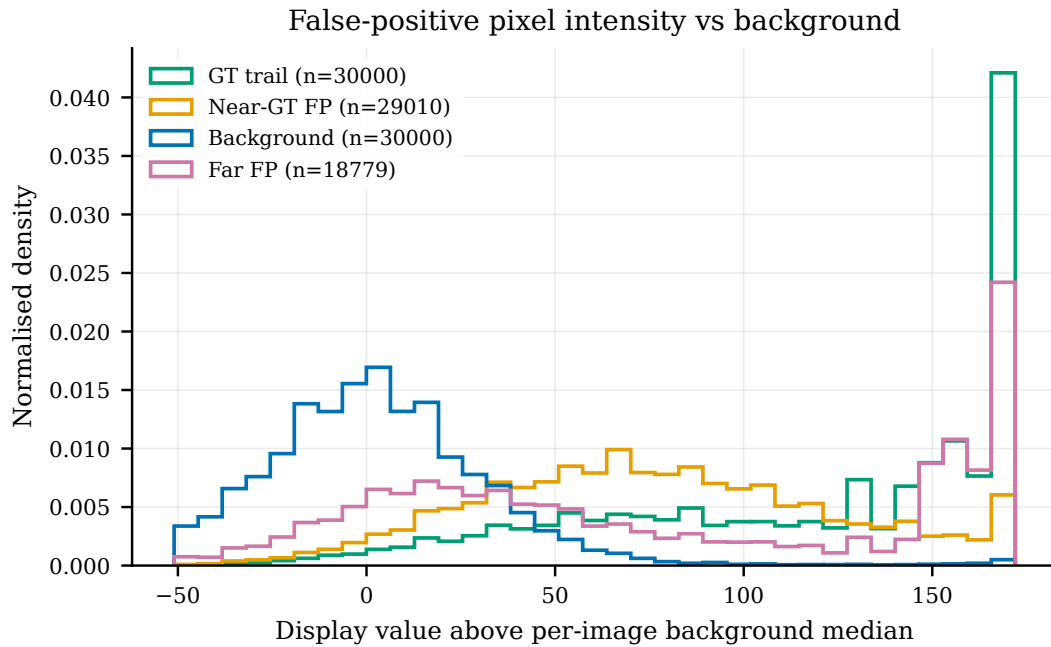


Figure A.3: Display-intensity distributions for false-positive, labelled-trail, and background pixels; the comparison is descriptive rather than evidence of label error.

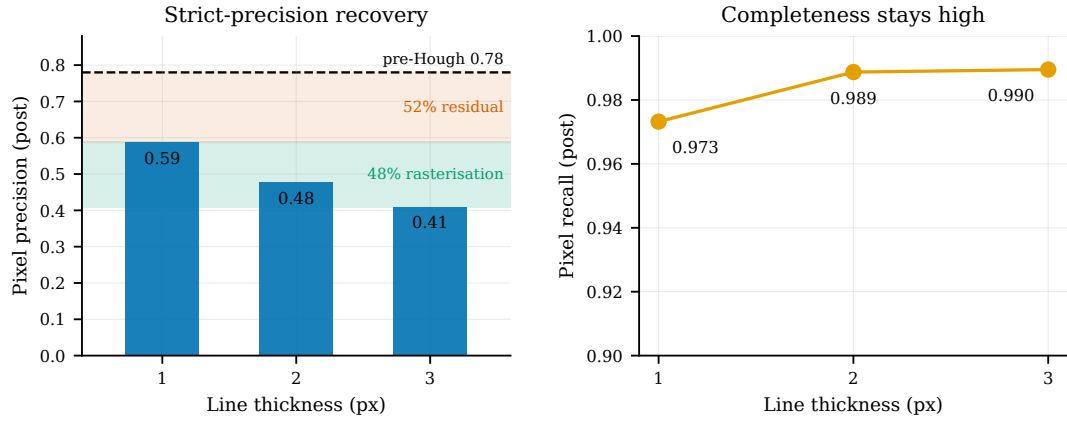


Figure A.4: Parity-canvas post-Hough precision and completeness under 1–3 px line-drawing thickness; the sweep bounds the effect of the drawing convention.

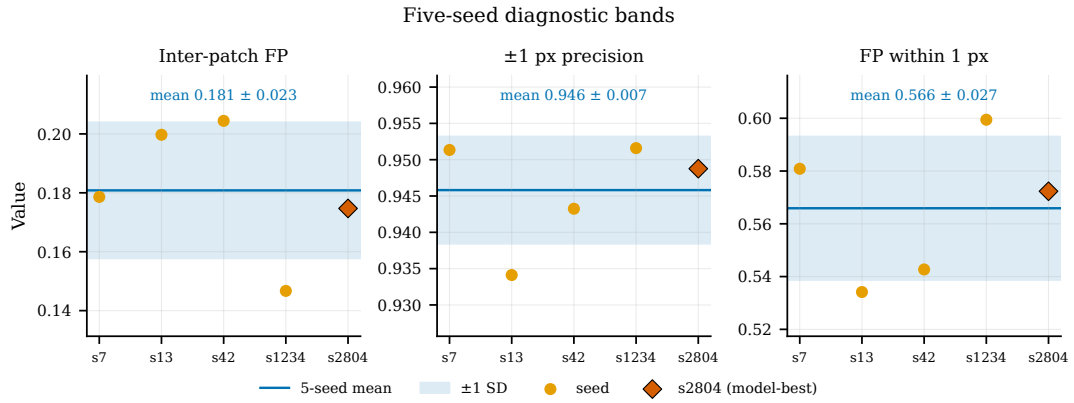


Figure A.5: Five-seed stability of the boundary checks: inter-patch false-positive share, ± 1 px precision, and false positives within 1 px. The diamond marks the selected seed-2804 winner.

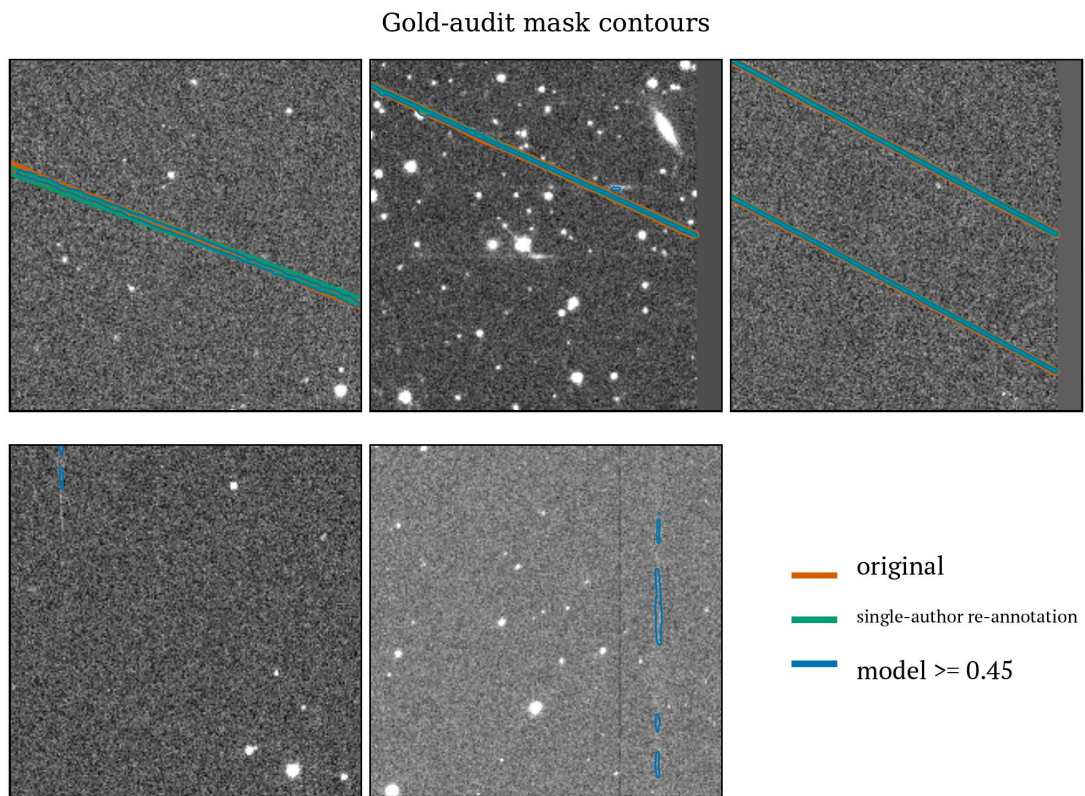


Figure A.6: Blinded re-annotation overlays for five audit crops; contours show original mask, re-annotation, and model prediction. The single-author re-annotation is not an independent ground-truth mask.

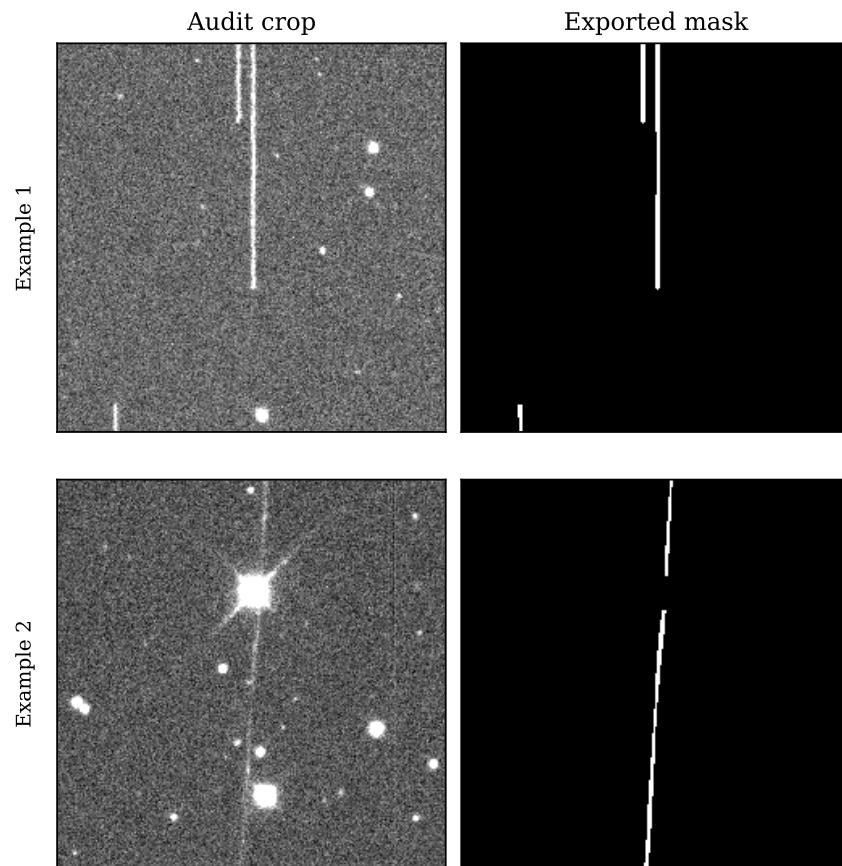


Figure A.7: Example sealed audit crops and exported binary re-annotation masks. Re-annotations used fixed zoom, contrast, and brush settings.

A.2 Supplementary Tables

Table A.1: Optuna hyperparameter search space.

Parameter	Type	Range / values	Notes
<code>learning_rate</code>	log-uniform	$[10^{-4}, 10^{-3}]$	Common range for all batch sizes
<code>bce_weight</code>	uniform	$[0.2, 0.8]$	$\lambda_{\text{Dice}} = 1 - \lambda_{\text{BCE}}$
<code>batch_size</code>	fixed allocation	$\{8, 16, 32\}$	45 trials total; 15 per batch size

Table A.2: Five-seed test means for the base and attention-gated U-Nets.

Metric	Base U-Net mean	Attention U-Net mean	Delta
Precision	0.847	0.848	+0.001
Recall	0.923	0.919	-0.005
F_1	0.884	0.882	-0.002
Dice	0.884	0.882	-0.002

A.3 Prediction Galleries

FP / TP patch gallery: green = GT, magenta = prediction (overlap = magenta inside green)

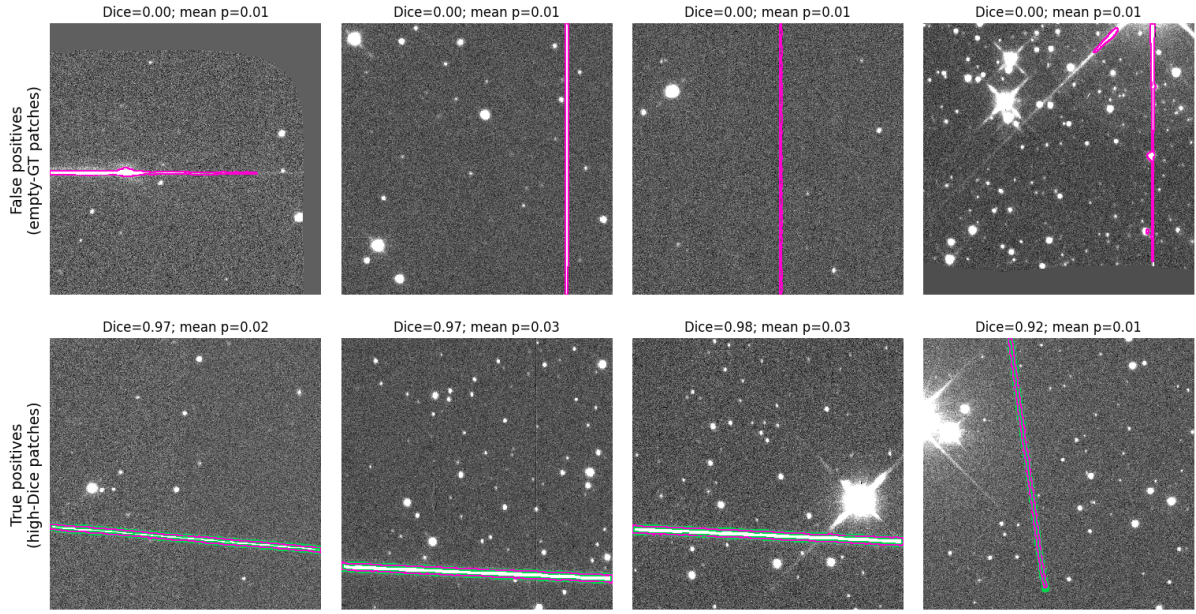


Figure A.8: False-positive and true-positive gallery for the selected winner, including bright-star diffraction-spike examples alongside high-Dice cases where bright stars are correctly ignored.

FP / TP patch gallery: green = GT, magenta = prediction (overlap = magenta inside green)

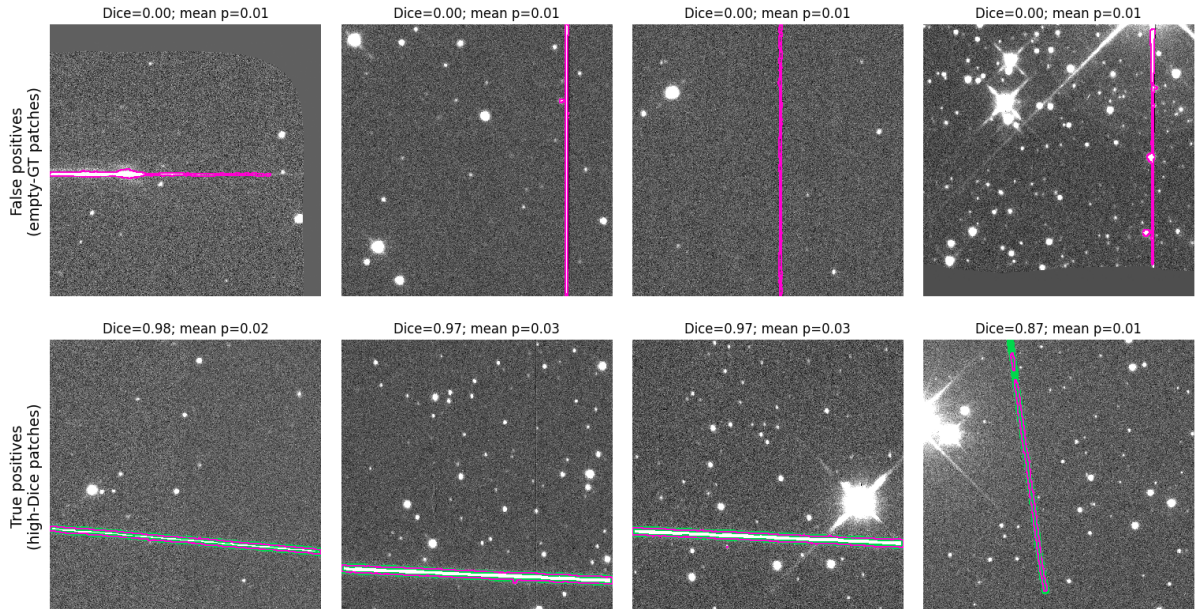


Figure A.9: The same eight patches for the attention-gated U-Net, in the order of Figure A.8, each with that model's own segmentation and Dice score.

Appendix B

Use of Auto-generation Tools

I have used Codex (ChatGPT) and Claude Code to support me across this work, primarily in the following areas:

- Providing advice and usage support for libraries like PyTorch and PIL.
- Generating boilerplate code for plotting, testing and scripting.
- Generating function docstrings.
- Reviewing and auditing project structure for completeness, consistency, accuracy and best practices in software engineering.